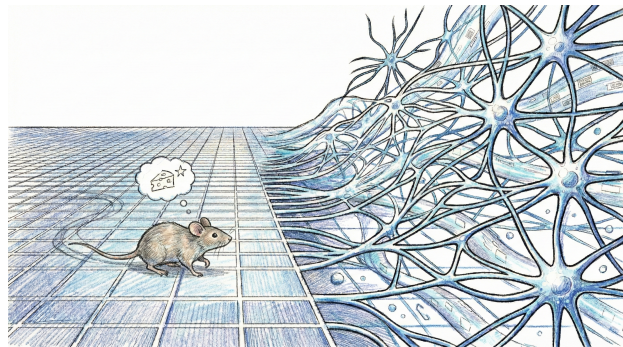




Beyond the Euclidean Map: A Reinforcement Learning Perspective on Hippocampal Representations

Francisco Nunes Figueiredo Neves da Silva

2nd Cycle Integrated Project in

Biomedical Engineering

Supervisor(s): Prof. Gonzalo Polavieja

Prof. Rita Gouveia Nunes

Ms. Carolina Gonçalves

Examination Committee

Member of the Committee: Dr. Tiago Costa

January 2026

Resumo

Tradicionalmente visto como um mapa Euclidiano fixo, coloca-se cada vez mais a hipótese de o hipocampo representar um espaço de estados flexível. Para explorar este conceito, investigámos de que forma a incorporação de características não espaciais na definição de estado de um Processo de Decisão de Markov influencia a aprendizagem. Modelámos o protocolo comportamental de Jercog et al., uma tarefa caracterizada por uma dinâmica de objetivos estocástica e estruturas de recompensa incertas, para testar se representações de estado aumentadas com contexto conseguem resolver eficazmente as restrições mnemónicas e a incerteza inerentes à tarefa. Esta abordagem permite-nos correlacionar a cinemática de agentes artificiais com futuras gravações de cálculo na região CA1 do hipocampo. Utilizando um ambiente sintético Gridworld, extraímos políticas de controlo ótimas para identificar assinaturas algorítmicas que se assemelhem às estratégias comportamentais desenvolvidas pelos roedores.

Avaliámos um espectro de arquiteturas de aprendizagem por reforço (AR), desde a Representação Sucessora (SR), que prevê a ocupação futura de estados, até políticas de Deep RL e de Entropia Máxima. Os agentes de Deep AR foram explicitamente concebidos para incorporar vetores de objetivo como entradas não espaciais. De forma semelhante, utilizando estes vetores de objetivo, a estrutura de Entropia Máxima distinguiu-se por suavizar a política, permitindo-nos modelar a formação de hábitos rígidos enquanto o agente equilibra a exploração com uma estratégia robusta. Os nossos resultados revelaram que: o motor preditivo da SR desestabilizou em regimes de objetivos em rápida mudança. Em contraste, os agentes DQN, UVFA-DQN e SQL, que codificam explicitamente vetores de objetivo, replicaram com sucesso o fenótipo comportamental observado nos roedores, tendo o DQN demonstrado um desempenho superior entre agentes AR. Isto sugere que a navegação em tarefas dinâmicas depende da codificação explícita do contexto do objetivo, em vez da tentativa de prever transições de estado específicas. Propomos que o hipocampo possa, de forma semelhante, integrar vetores de objetivo não espaciais diretamente no seu código neuronal, uma hipótese que pretendemos validar biologicamente com gravações fisiológicas em trabalhos futuros.

Palavras-chave: Navegação Espacial, Aprendizagem por Reforço, Hipocampo, Representação Generalizada de Estados.

Abstract

Traditionally thought to encode a fixed Euclidean map, the hippocampus is increasingly hypothesized to represent a flexible state space. To explore this, we investigated how incorporating spatial and non-spatial features into the Markov Decision Process state definition influences learning. We modeled the behavioral protocol from Jercog et al., a task characterized by stochastic goal dynamics and uncertain reward structures, to test if context-augmented state representations can effectively solve the mnemonic constraints and uncertainty inherent to the task. This design enables us to correlate the kinematics of artificial agents with future hippocampal CA1 calcium recordings. Using a synthetic Gridworld suite, we first extracted optimal control policies to identify algorithmic signatures that resemble the behavioral strategies developed by rodents.

We benchmarked a spectrum of reinforcement learning (RL) architectures, contrasting the Successor Representation (SR), which predicts future state occupancy, against Deep RL and Maximum Entropy policies. Deep RL agents were explicitly designed to incorporate goal vectors as non-spatial inputs. Similarly leveraging these goal vectors, the Maximum Entropy framework distinguished itself by softening the policy, allowing us to model the formation of rigid habits as the agent balances exploration against robust strategy. Our results revealed that: the predictive engine of the SR destabilized in regimes of rapidly shifting goals. In contrast, DQN, UVFA-DQN and SQL agents, successfully replicated the behavioral phenotype observed in rodents, with DQN specifically achieving superior performance among RL agents. This suggests that navigation in dynamic tasks relies on explicitly encoding goal context rather than attempting to predict specific state transitions. We propose that the hippocampus may similarly integrate non-spatial goal vectors directly into its neural code, a hypothesis we aim to biologically validate against physiological recordings in future work.

Keywords: Spatial Navigation, Reinforcement Learning, Hippocampus, Generalized State Representation.

Contents

Resumo	iii
Abstract	v
List of Tables	ix
List of Figures	xi
1 Introduction	1
1.1 Motivation	3
1.2 Problem Statement	4
1.3 Objectives and Deliverables	5
2 Background	7
2.1 Mechanisms of Adaptive Behavior	7
2.1.1 The Duality of Behavioral Control: From Reflexes to Stochastic Planning	7
2.1.2 Stochasticity and Exploration in Animal Foraging	8
2.1.3 Internal Simulation as the Engine of Flexibility	9
2.2 Reinforcement Learning	10
2.2.1 Model-Free Learning: From Synaptic Caching to Replay Dynamics	10
2.2.2 Model-Based Learning: Simulation and the Cost of Insight	11
2.2.3 Successor Representation: From Static Maps to Predictive Horizons	12
2.3 Beyond Space: Latent Representations and Generalization	14
3 Methods	15
3.1 Dataset	15
3.2 Gridworld Simulation Suite	16
3.3 Reinforcement Learning	17
3.3.1 Model-Free Algorithms	19
3.3.2 Model-Based Algorithms	25
3.3.3 Successor Representation Algorithm	26
3.3.4 Performance Metrics	27
4 Results	30
4.1 Expected Results	30

4.1.1	Maximization Bias and Stability: The Jensen Gap	30
4.1.2	Variance Reduction in On-Policy Control: Expected SARSA	31
4.1.3	Balancing Learning Speed and Noise: n -Step SARSA	31
4.1.4	Soft Optimality Improves Exploration: Soft-Q Learning	32
4.1.5	Deep Representation and Goal-Conditioned Generalization	32
4.1.6	Model-Based Planning and Uncertainty-Driven Exploration	33
4.1.7	Cognitive Mapping and Latent Learning: Successor Representation	33
4.2	Preliminary Results	34
5	Conclusions and Future Directions	39
	Bibliography	40
A	Gridworld Behavioral Analysis	47
B	Agent Performance Benchmarking	49
C	Model Parameters	53
D	Schedule Plan	57
E	Implementations	58

List of Tables

3.1	Gridworld Simulation. Classification of the seven environmental conditions based on goal stability, sub-goal dynamics and reset conditions. Checkmarks indicate the active features for each environment.	17
C.1	Global Simulation Parameters. Configuration applied across all experimental runs. . . .	53
C.2	Agent-Specific Hyperparameters. Configuration for Tabular and Deep MF RL agents. .	54
C.3	Agent-Specific Hyperparameters. Configuration for Dyna-Q, Dyna-Q (UCB) and SR agents.	55

List of Figures

- 4.1 **Performance Metrics in Environment 7.** Quantitative analysis of MF, MB and SR agents over 4,000 episodes, with random goal relocations every 1,000 episodes. (*Top-Left*) Final Average Reward: Measures asymptotic proficiency by averaging rewards over the final 100 episodes; higher is better. (*Top-Mid*) Total Cumulative Reward: Proxy for total regret, distinguishing fast learners from slow learners based on the total cost of the learning process. (*Top-Right*) Convergence Ep: The first episode index where the agent consistently achieves 90% of maximum reward; lower indicates faster learning. (*Bottom-Left*) Optimality Gap: Ratio of steps taken versus the optimal Manhattan distance; values > 1.0 indicate inefficient loops or detours. (*Bottom-Mid*) Stability: Standard deviation of rewards at the last 100 episodes; lower values indicate a consistent policy. (*Bottom-Right*) Recovery Time: Mean episodes required to secure a positive reward immediately following a goal switch, measuring plasticity. Error bars represent the standard error of the mean across $n = 3$ independent trials. Note the large variance in Recovery Time and Convergence for SR and DQN, indicating that performance differences in these specific metrics may lack statistical significance due to the high stochasticity of the trigger mechanism. 35
- 4.2 **Comparative Learning Kinematics: Rodents and Artificial Agents.** *Top Row:* Behavioral tracking data from mice ($n = 23$), adapted from Jercog et al. [20]. *Bottom Row:* Simulation results for all agent architectures in Environment 5. **(A)** Evolution of task success: Ratio of correct trials (*Top*) and Cumulative Reward (*Bottom*). **(B)** Search Efficiency: Time (*Top*) or steps (*Bottom*) required to locate the stochastic trigger zone. **(C)** Navigation Efficiency: Time (*Top*) or steps (*Bottom*) required to travel from the trigger zone to the goal. Data represents Environment 5 with a total of 4,000 episodes. Shaded regions indicate 95% confidence intervals (*Top*) or standard deviation across 3 independent runs (*Bottom*). 37
- A.1 **MF, MB and SR Kinematics: Search and Navigation.** Comparative analysis of behavioral components in Environments 4, 6 and 7. (A) Search Phase: The number of steps required to locate the trigger zone from the start position. (B) Goal Phase: The number of steps required from the trigger zone to the active reward location. The reward relocates among the four corners every 1000 episodes. Plots reflect data averaged over 3 independent runs with shaded regions indicating standard deviation. 48

B.1 **Comparative Learning Dynamics of MF, MB, and SR Agents.** Temporal evolution of cumulative reward against MF, MB and SR architectures across distinct environmental configurations. Plots show the moving average of rewards over 4000 episodes with a active reward switch interval of 1000 episodes, averaged across 3 independent runs, with shaded regions indicating standard deviation. 50

B.2 **Quantitative Performance Metrics for MF, MB and SR Agents.** Performance metrics extracted from learning sessions across Environments 1 through 6, benchmarking the distinct control policies of MF, MB and SR architectures. Error bars represent standard error across 3 independent runs. 51

D.1 **12-Month Research Schedule.** **Green:** Completed tasks. **Orange/Red Marks:** Evaluation Deadlines. **Yellow:** Ongoing tasks. **Red:** Future tasks. 57

Chapter 1

Introduction

Consider the act of navigating a complex subway system, such as the London Underground. To travel effectively, a geographically precise map, one that preserves exact distances and angles, is surprisingly inefficient. Instead, commuters rely on a topological schematic that distorts reality to highlight what matters: connectivity, sequence and flow. This reveals a fundamental biological constraint: without the privilege of an internal global positioning system, the brain is forced to treat space as a sequence of events rather than a coordinate grid [1]. Lacking absolute metrics, it must organize experience temporally, constructing chains of neural activity to infer trajectories from a continuous stream of egocentric sensations [1–4]. To build a usable map from this ambiguous input, the brain must find the underlying pattern: it groups related moments together and keeps distinct events separate, turning a messy stream of sensations into a connected path [1, 2]. This convergence of navigation and memory is centrally managed by the hippocampus. By binding past experiences to the spatial locations where they occurred, the hippocampus constructs a flexible internal map. This structure enables the agent to move beyond simple reflexes by providing the foresight needed to navigate complex environments and plan a path toward a specific goal [1, 5–7].

For decades, the dominant metaphor for how the brain achieves this has been O’Keefe and Nadel’s cognitive map [8]. This classical view posits that the hippocampus functions as a geometric coordinate system, where individual neurons, known as place cells, exhibit spatially restricted firing patterns called place fields. Collectively, these place fields are thought to furnish an explicit map of the environment. However, the hippocampus is not merely a GPS; it is the seat of memory [9–12]. Memories are integral to how we understand the world and plan future actions, and neurophysiologically, they are formed by binding events that occur in close temporal proximity. Consequently, neural activity associated with retrieval is not a static snapshot of a past location, but encompasses information about what the animal might experience next. This predictive capacity is essential because a hallmark of adaptive behavior is the effective use of experience to maximize reward, even when actions are separated from their consequences in both space and time [1, 10, 13]. If the ultimate function of memory is to guide decision-making, this suggests an alternative perspective: that hippocampal representations may not be limited to static spatial coordinates, but could encode the predictive transition structure of the environment.

To rigorously formalize how these internal representations guide adaptive action, we turn to reinforcement learning (RL). In this framework, the concept of state extends the definition of a simple Euclidean location to encompass the full context required for decision-making. RL addresses the theoretical problem of how an agent, biological or artificial, should traverse these states to maximize expected future rewards, formally known as value, without being explicitly told which actions to take [13]. These models have become indispensable tools for identifying potential mechanisms underlying goal-directed behavior, bridging the gap between rapid control in artificial systems [14, 15] and the behavioral patterns observed in animals [16, 17]. Within this framework, we can see that the brain likely houses parallel control systems, ranging from computationally expensive planning to efficient habits. This prompts a question regarding the neural implementation of these algorithms: how must the hippocampus encode these states to support flexible behavior without incurring the computational burden of full simulation?

To support flexible behavior, the hippocampus must go beyond encoding physical location by constructing a generalized state representation that captures the structure of the task. This need challenges the traditional view of place fields as fixed geometric markers. Instead, theoretical neuroscience offers a spectrum of frameworks to explain the emergence of these fields, particularly when they encode non-spatial variables like context or goals. Some models propose that place fields actually reflect the topological connectivity of the environment, creating a schematic that highlights accessible sequences rather than metric distance [18]. Others suggest that these fields arise from contextual inference, where neurons represent hidden states to ensure that identical sensory cues are mapped to distinct fields depending on the current rule [1, 19]. Despite their differences, these frameworks share a common intuition: that the cognitive map is organized by predictive relationships, not just physical proximity. We begin with the Successor Representation (SR) as a practical model to test our theoretical framework against real data. By formalizing the concept of future occupancy, the SR provides a concrete mechanism for how predictive place fields might bind spatial locations with non-spatial contexts. We prioritize this framework because it retains the rigorous convergence properties of RL, allowing us to benchmark these state representations against biological behavior before exploring more latent-state models.

We ground our investigation in the experimental paradigm of Jercog et al. [20], selected because its dynamic goals and mnemonic demands force the animal to move beyond simple geometric navigation. Instead of a static map, this task requires a flexible state representation that binds spatial coordinates with changing contexts. By benchmarking RL algorithms on a digital replica, we first isolate the control strategies that mimic biological behavioral strategies. We then examine the internal states of these agents, proposing that effective navigation relies on a generalized state representation that integrates geometry with non-spatial variables, such as topological connectivity, prediction and the sequential organization of episodic memories. Finally, we validate this framework by correlating these extracted state representations with hippocampal recordings, testing whether the neural code reflects the full structure of the task rather than just physical location.

1.1 Motivation

Navigation is more than just locomotion, it is a cognitive challenge that requires an animal to organize fragmented sensory experiences into a coherent plan of action [1]. Since the original definition of the cognitive map [21] and the subsequent discovery of spatially tuned place cells [8], the hippocampus has been canonically viewed as the brain’s metric positioning system. This framework proposes that hippocampal activity furnishes an explicit, geometric representation of space, enabling animals to compute vectors and distances essential for survival. However, this strictly spatial view cannot fully explain how animals actually navigate: if the hippocampus is strictly a spatial chart, how do we account for its critical role in non-spatial domains, such as the organization of time [2–4], the binding of stimulus-response associations [22, 23] and the maintenance of abstract rule sets [6, 12, 19]? The persistence of these non-spatial signals suggests the *map* metaphor is too limited. Instead of a simple chart, these findings point toward a flexible, generalized state representation. The hippocampus appears to bind spatial coordinates with abstract task variables, raising the critical question of how such a complex structure is actually encoded in the brain.

We include the SR within our RL benchmark because it challenges the traditional view of the cognitive map as a purely geometric entity, reframing it instead as a predictive engine based on expected future occupancy [18]. This is particularly relevant for the stochastic dynamics of Jercog’s et al. task [20] because it computationally decouples the environmental structure from the reward function. This separation allows an agent to adapt to shifting goals by simply updating value vectors, without the need to relearn the underlying topology, a flexibility that may mirror the efficient behavioral strategies observed in mice. However, the SR serves as a computational baseline rather than a final solution. Our broader investigation will subsequently benchmark its extracted state representations against alternative mathematical formulations, such as spectral graph theory and probabilistic inference [1, 19, 24]. These complementary frameworks will be essential for extracting latent state representations that specifically highlight topological connectivity and the associative structure of episodic memories, features that may offer a more accurate description of the hippocampal representations.

RL offers a complementary framework for investigating these generalized state representations, particularly regarding how temporal horizons are encoded within the neural substrate. This approach is motivated by a structural parallel between algorithmic parameters and hippocampal anatomy. Specifically, the longitudinal axis appears to mirror the function of the discount factor (γ): studies by Contreras et al. [25] suggest this axis encodes a gradient of temporal scales, ranging from the dorsal region, which optimizes for immediate consequences, to the ventral region, which prioritizes long-term outcomes. This correspondence implies that the mathematical concept of temporal discounting may be physically implemented within the tissue itself. Benchmarking RL agents allows us to test if this biological architecture is truly necessary for solving complex planning tasks.

This biological gradient supports a form of mnemonic scaling [9], enabling the brain to process the world at different resolutions simultaneously, handling immediate sensory details in the dorsal region while planning over long-term horizons in the ventral region. We hypothesize that this multi-scale archi-

ture might provide a certain mathematical structure for a hierarchical generalized state representation, where complex tasks are decomposed into reusable sub-routines to facilitate bootstrapping during learning [1, 11]. Given the uncertainty of the invisible trigger zones in the Jercog task [20], such a hierarchy allows the agent to adapt efficiently. We will discuss this task organization further in later sections, but essentially, this hierarchy enables the transfer of learned structural rules to new sessions without relearning the map from scratch. We investigate whether this hierarchy emerges spontaneously in our agents, offering a computational explanation for how the hippocampus generalizes across shifting contexts.

To address these limitations, we adopt the high-throughput protocol from Jercog et al. [20], as its dynamic structure forces the animal to abandon rote memorization in favor of flexible planning and eventual clear search heuristics. This complexity allows us to first identify the artificial agents that derive optimal control policies. Motivated by some empirical evidence that hippocampal codes are intrinsically hierarchical, predictive and highly connected, we then scrutinize the internal state representations of these best-performing agents. By applying a several analytical frameworks, we aim to determine if these state representations capture both spatial and non-spatial features, thereby corroborating the existence of a flexible, generalized representation.

1.2 Problem Statement

The study of adaptive behavior has long been dominated by a fundamental dichotomy regarding how biological agents control their actions. On one hand, the teleological model posits that behavior is purposive: an animal acts based on a cognitive map of instrumental relations [26], dynamically computing the value of a goal to guide its decisions. On the other hand, the mechanistic stimulus-response model [27] argues that behavior often consolidates into rigid habits, automated routines triggered by environmental cues independent of the current value of the outcome. This distinction is not merely theoretical, but can be empirically dissected using reward revaluation procedures. As demonstrated by Dickinson et al. [28], a rat trained to press a lever for food will initially stop pressing if the food is devalued, e.g., paired with nausea, indicating that the behavior is a sensitive, goal-directed action. However, with extended training, the animal persists in pressing the lever even when the reward is worthless, revealing a transition to behavioral autonomy where the action has solidified into a habit.

This biological duality finds its rigorous mathematical counterpart in RL, which partitions algorithms into two distinct families: model-free (MF) and model-based (MB). MF algorithms are the computational equivalent of habits [29]. Rather than learning the causal rules of the environment, these agents simply memorize which actions yielded rewards in the past. This approach is highly efficient allowing for rapid, reflex-like decisions, but it renders the agent blind to the underlying structure of the task. Much like the over-trained rat that continues to press a lever for devalued food, a MF agent relies on cached history rather than current prediction.

Conversely, MB algorithms computationally model the teleological system [30, 31]. Analogous to an animal consulting a cognitive map, these agents build an internal model of how the world works,

allowing them to run *mental simulations* of potential futures before acting. This method requires more computation, but it allows the agent to handle changes much more effectively. Because they understand the linkage between states, MB agents can instantly adapt to sudden changes, such as a blocked route or a shift in goal value, transferring their knowledge to new contexts without the need for the extensive retraining required by MF agents.

However, both frameworks struggle in dynamic environments. While MB planning dominates in constrained domains like Chess or Go [32], it becomes computationally intractable in the real world where possible futures explode exponentially. In contrast, MF learning is too rigid [33], failing to adapt quickly when goals shift. This limitation challenges the view of the hippocampus as a static map. Instead, its representations must be dynamic and predictive to bridge the gap between rigid habits and complex planning.

We base our approach on RL, which provides the standard framework for distinguishing between rigid habits and flexible planning. Hence, this allows us to extract control policies and directly compare them with the behavioral strategies observed in mice. However, achieving behavioral alignment does not guarantee representational fidelity. Within RL, the SR attempts to bridge this gap by offering a predictive map, yet it faces critical limitations. Its learned representations often exhibit spatial distortions, and its efficacy frequently relies on hybridization with MB based planning, which undermines the argument that a predictive map alone suffices for flexible behavior [19]. Consequently, to capture the true nature of the hippocampal code, we must extend our investigation beyond RL. We aim to explore whether more robust mathematical formulations, such as graph representations and probabilistic inference models can yield the stable, biologically plausible representations required to withstand the uncertainty and mnemonic demands of the Jercog task [20].

1.3 Objectives and Deliverables

The main goal is to understand the structure of state representations by anchoring our proposed mechanisms in concrete behavioral evidence. We establish RL as our foundational tool to model the spectrum of control strategies, ranging from reactive systems to complex planning architectures. Leveraging the dataset from Jercog et al. [20], our immediate priority is to benchmark these algorithms to identify the control policies that best replicate biological behavior. This validation lays the groundwork for our investigation: we aim to construct a state representation that integrates spatial geometry with non-spatial context. To this end, we employ Deep RL agents to capture how dynamic goals, subgoals and other task rules are encoded, while simultaneously testing the specific predictive engine hypothesis of the SR, which organizes the state space based on expected future occupancy. Ultimately, we benchmark these RL-derived models against alternative frameworks, such as graph representations and probabilistic inference, to determine which formalism best reconstructs the biological cognitive maps observed in CA1 dorsal region recordings.

Goal 1: Simulating the Task to Identify Optimal Control Policies

To establish a computational baseline for optimal control, we will engineer a synthetic Gridworld

suite that functions as an *in silico* replica of the biological task. Parameterized by the specific stochastic constraints and transition dynamics extracted from the behavioral dataset, we will deploy RL agents across a gradient of computational complexity. Ranging from MF and MB heuristics to predictive SR models, our objective is to determine which architecture most robustly converges on optimal control. This will allow us to identify the specific RL agents that best approximate the subject's behavioral strategies.

Goal 2: Aligning Optimal Control Policies with Rodent Trajectories

To validate the biological plausibility of our computational models, we will perform a quantitative comparison between the RL agents developed in Goal 1 and the rodent tracking data. Our goal is not to identify the agent that maximizes reward, but the one that maximizes *behavioral fit*. Using metrics like path efficiency and decision latency, we aim to identify which RL model best matches the rodents' actual, and often sub-optimal, navigation patterns. This comparative analysis allows us to reverse-engineer the animal's strategy, distinguishing whether observed inefficiencies are mere errors or purposeful deviations driven by specific algorithmic constraints, such as the exploration bonuses found in maximum entropy models.

Goal 3: Extracting and Benchmarking State Representations

Having identified the RL agents that best replicate biological behavior, we will extract and analyze their latent state representations. For Deep RL agents, this involves probing the hidden layers for emergent spatial and non-spatial selectivity, effectively isolating *synthetic place fields* from the network's internal activity. For the SR, we specifically test the hypothesis that the columns of the expected future occupancy matrix map directly onto biological neural assemblies. We will then benchmark these internal representations against established theoretical frameworks, such as spectral graph theory and probabilistic inference models. This analysis aims to determine which computational formalism best captures the emergence of cognitive maps in a task characterized by dynamic goals and stochastic sub-goals, irrespective of the behavioral strategy used.

Goal 4: Biological Validation of State Representations

Finally, we seek to validate our extracted state representations against physiological reality. We will compare the state representations generated by our diverse mathematical frameworks, ranging from the latent hidden states of Deep RL agents to spectral graph eigenvectors and probabilistic inference models, against large-scale neural recordings from the CA1 dorsal hippocampus. This analysis extends beyond traditional spatial coding to investigate whether the biological data encodes non-spatial information, such as context-dependent trigger zones and dynamic rule switches. By quantifying the representational alignment between neural manifolds and our models, we aim to determine if the hippocampal code is best described as a static spatial map or a flexible state-space optimized for cognitive control.

Chapter 2

Background

2.1 Mechanisms of Adaptive Behavior

2.1.1 The Duality of Behavioral Control: From Reflexes to Stochastic Planning

The study of behavior has long been divided by a conflict between two opposing views of animal cognition. On one side, the mechanistic model [27, 34], proposes that behavior is essentially a reflex: a direct, automatic reaction to a trigger in the environment. This model aligns with the classical concepts of fixed action patterns and conditioned responses. The intuitive nature of this framework makes it the common default, often masking the agent's true internal dynamics.

This mechanistic perspective finds its empirical origin in Thorndike's puzzle box experiments [27]. When placed in a box, a cat would initially react with ineffective, indiscriminate clawing. Eventually, and unintentionally, the animal would trigger the release mechanism obtaining freedom and food. Thorndike observed that with repetition, the satisfying outcome reinforced the specific action required to escape, while ineffective movements were gradually eliminated. Crucially, he argued that this learning was driven by the *Law of Effect* [34]: the creation of a rigid association between a stimulus, e.g. the box, and a response, e.g. pressing the lever. In this view, no speculation about the mind or future goals was necessary, as the animal was simply executing a stored routine triggered by the environment.

Recent work by Corbit et al. [35] has further dissected the conditions that promote this behavioral autonomy. Their experiments demonstrated that the presence of discrete, antecedent stimuli, such as a light signal predicting lever availability, accelerates the formation of habits. Animals trained to respond to these external cues rapidly became insensitive to outcome devaluation, a hallmark of habitual control [36]. In contrast, animals that retained control over the lever's insertion, i.e. acting without a predictive cue, remained goal-directed. This indicates a clear distinction in the mechanistic view: when an agent lacks autonomy and relies on environmental triggers, behavior favors stimulus-response habits [35].

However, this reflexive model struggles to account for navigation in complex, open environments. While simple T-mazes [37] allow an animal to rely on binary, egocentric turns, tasks like the Morris Water Maze [38] demand a higher order of state representation. In this paradigm, a rat must locate a platform hidden beneath opaque water. Lacking local sensory beacons such as sight or smell of the goal, the

animal cannot rely on a simple stimulus-response chain. Instead, accurate directionality requires the continuous monitoring of the animal's position relative to extra-maze cues. As noted by O'Keefe and Nadel [8], successful performance in such tasks implies the existence of a cognitive map, an internal, allocentric representation of space that allows the agent to flexibly infer routes across the environment rather than merely responding to a sequence of triggers.

In contrast, the teleological model [26] offers a more sophisticated account, asserting that certain behaviors occur not because they are triggered by a stimulus, but because they are controlled at the moment of execution by the animal's knowledge of the consequences. To distinguish these goal-directed activities from rigid responses, we can classify them as actions [28]. This distinction is not merely semantic, because a teleological system gives the animal superior behavioral flexibility. Unlike a habit, which persists regardless of outcome, a goal-directed action allows an animal to adjust its behavior immediately in response to changes in the value of a goal, whether driven by a shift in metabolic state, e.g., satiety, or new knowledge about the reward's quality. This theoretical framework rests on Tolman and Honzik's classic latent learning experiments [39]. They observed that rats navigating a maze without food appeared to wander without direction. However, when a reward was introduced on the 11th day, these animals demonstrated near-instantaneous mastery of the route. This sudden shift seemed to reveal that the animals had built a flexible cognitive map during their unrewarded exploration but simply lacked the motivation to use it until the goal value changed. Tolman further identified that during uncertain moments, rats would pause and look back and forth between options, a behavior he termed *vicarious trial and error* suggesting an internal deliberation or simulation of future possibilities [39, 40].

Decades later, modern electrophysiology has materialized these cognitive constructs into observable neural events. Pfeiffer and Foster [41] provided evidence that this decision-making is driven by a planning mechanism in the hippocampus. By recording large populations of place cells while rats navigated an open arena, they discovered that during brief pauses, the brain spontaneously generated rapid sequences of neural activity. In fact, these *trajectory events* did not merely reflect where the animal had been but rather projected forward, accurately tracing the specific path the animal was *about to take* to reach its goal. This neural look-ahead mechanism proved highly flexible, corroborating the definition of an action by Dickinson [28]. The recorded sequences predicted the rat's future behavior regardless of whether it was heading toward a known "Home" location or exploring a random site in the arena. Furthermore, the system demonstrated the capacity to compute flexible shortcuts, simulating trajectories between start and end points that had never been directly traversed before. This supports the view that the hippocampus serves not merely as a repository for past memories, but as a computational engine that actively constructs spatial plans to guide adaptive decision-making.

2.1.2 Stochasticity and Exploration in Animal Foraging

Although classical experiments successfully distinguished between habits and actions, they relied on static, predictable environments that do not reflect the uncertainty of the real world. To investigate decision-making under uncertainty, modern neuroscience has turned to the *dynamic multi-armed bandit*

task [13]. In this paradigm, subjects must choose between multiple ports, each offering a probability of reward that drifts over time.

Analysis of rodent behavior in these tasks reveals that optimal performance is not a fixed, deterministic sequence but a continuous modulation between exploration and exploitation. As demonstrated by Beron et al. [17], mouse behavior is characterized by distinct regimes of *stickiness*, i.e. a bias toward repeating past choices, and stochasticity. Unlike a theoretical *ideal bayesian observer* that perfectly calculates the probability of a switch based on all available evidence, mice employ a more heuristic strategy. They exhibit high-fidelity exploitation of known good ports but switch to exploration in a probabilistic manner. Importantly, this stochasticity imposes a fundamental limit on how well mathematical models can predict individual choices. However, it is not a failure of intelligence. Rather, this *sticky stochasticity* [17] represents an efficient solution for volatile environments, allowing the agent to preserve high reward rates while minimizing the cognitive cost of constantly recalculating exact probabilities.

This behavioral variability is a response to the uncertainty the animal faces. Webb et al. [42] distinguish between expected uncertainty, i.e. noise inherent to a stable reward source, and unexpected uncertainty, i.e. a fundamental change in the environment's rules. To manage this, animals must actively gather data, a process Johnson et al. describe as *Information Foraging* [43]. Just as Tolman's rats engaged in *vicarious trial and error* [39] to sample future outcomes, Johnson et al. propose that the hippocampus actively samples memory and space to resolve ambiguity. This suggests that exploration is not merely random motion, but a directed effort to reduce uncertainty about the current state.

Consequently, to model biologically plausible navigation, we must move beyond rigid, greedy algorithms. The empirical evidence suggests that biological agents operate under a principle of soft optimality, seeking to maximize rewards while simultaneously maintaining high entropy in their policy to ensure continued exploration. This concept aligns with the computational framework of maximum entropy control [44] and soft Q-Learning (SQL) [45], which penalize deterministic policies to prevent the agent from becoming trapped in local optima. We will return to this distinction between hard and soft optimality in later sections, where it serves an interesting idea for our implementation of RL agents.

2.1.3 Internal Simulation as the Engine of Flexibility

How does the brain support this flexibility? To achieve goal-directed control, the brain must separate neural activity from the immediate environment, allowing the agent to create new plans and predict consequences. The emerging consensus suggests that this internal world model is physiologically instantiated by the hippocampus, which functions not merely as a memory store [9], but as a dynamic predictive engine [10, 18, 41].

Recent evidence shows that this predictive capacity relies on a dual coding scheme: neurons encode information not only through their firing rates but through the precise timing of spikes relative to the background brain rhythm. As described by George et al. [46], hippocampal place cells exhibit *theta phase precession*, a phenomenon where neurons fire progressively earlier in the theta cycle (8–10 Hz) [47] as the animal traverses a location. This temporal shift has a significant computational consequence:

within a single oscillatory cycle, lasting just over 100 milliseconds, the population activity compresses time, smoothly sweeping from a representation of the animal's immediate past to a simulation of its future trajectory.

This rapid temporal compression allows the brain to perform inference at the speed of synaptic plasticity. Penny et al. [48] propose that these compressed sequences function as distinct computational operations: forward sweeps, i.e. theta sequences, facilitate online decision-making and route planning, while rapid backward sweeps, i.e. replay, enable the offline consolidation necessary for learning. This framework suggests that the hippocampus operates as a generative model: when an animal pauses at a decision point, similar to *vicarious trial and error* events [39], the brain creates a *theta flickering* effect, rapidly cycling through alternative future maps to evaluate their value. Thus, the switch from rigid habit to flexible action is might be mediated by the recruitment of these neural assemblies, which momentarily decouple the animal from the present to simulate the consequences of the future.

2.2 Reinforcement Learning

2.2.1 Model-Free Learning: From Synaptic Caching to Replay Dynamics

One of the most robust convergences between artificial intelligence and neuroscience is found within the paradigm of MF RL. Engineered systems designed to optimize behavior in complex environments face the same challenge as biological agents: how to predict future rewards without an explicit instruction manual. To solve this, computer science developed the temporal difference (TD) algorithm [13, 29, 49], a method that allows a system to bootstrap its value estimates by comparing its current prediction to the reality of the moment. Originally inspired by animal behavioral data [50], this algorithm has emerged as the predominant neural theory of the phasic dopamine response [50–53].

It has long been suggested that the mammalian midbrain implements this computation directly [50, 54]. Since dopamine signals closely match the TD prediction error, the brain appears to track the difference between actual and predicted rewards. Anatomically, this process is thought to be orchestrated within cortico-striatal loops [14, 22, 55], where medium spiny neurons in the striatum encode the value of states or state-action pairs based on afferent inputs from the frontal cortex. The learning of this value function is driven by dopamine-mediated adjustments of the cortico-striatal synapses connecting these neurons.

But how is this error signal biologically constructed? In this circuit, dopaminergic nuclei receive inhibitory feedback from the striatum, i.e. the prediction, and direct excitatory inputs from sensory areas [14, 22] encoding consumed rewards such as the immediate gustatory signals from sugar solutions. By physically subtracting the predicted value from the experienced reward via these synaptic inputs, the dopamine neuron broadcasts a global error signal that updates the agent's policy.

This mechanism offers a computational substrate for the *Law of Effect* [34]: actions followed by a positive prediction error are reinforced, increasing the probability of their repetition. Indeed, neuroimaging provides distinct anatomical corroboration for this framework. MF value signals have been isolated

within the posterior putamen and the supplementary motor cortex [22, 56]. This localization mirrors the shift toward automatic behavior [57]. Activity in these regions peaks after extensive training, marking a transition to habitual control, a state where behavior persists even if the reward loses its value [36].

While this caching of values allows for rapid, reflex-like decision-making, does it explain the full range of biological intelligence? Clearly not. As demonstrated by Tolman’s classic latent learning experiments [21], animals often transcend this simple reinforcement principle, navigating via internal maps rather than mere history-dependent caches. MF learners are inefficient in changing environments because they lack state relationships, requiring the agent to physically re-experience the path to update values.

However, recent computational insights [14, 18, 53, 56, 58, 59] suggest the distinction between rigid caching and flexible planning may be more fluid than previously thought. How can a MF system achieve the flexibility of a planner? The answer may lie in the intersection of RL and memory consolidation. It has been argued that updating state-action values function in a MF manner, when combined with the replay of past experiences from a memory buffer, is mathematically equivalent to MB planning [58]. In this view, the replay events observed in the hippocampus [5, 10, 58] do not merely consolidate memory but actively generate training data for the striatal value system. By generating forward replay events, the brain can propagate value updates through the network offline, allowing a MF system to account for changes in the environment without immediate physical re-exploration. This suggests how replay allows agents to combine the efficiency of habits with the flexibility of a map.

2.2.2 Model-Based Learning: Simulation and the Cost of Insight

Challenging the mechanistic view, Tolman [21] proposed that animals are latent planners, capable of constructing internal cognitive maps independent of immediate reward. This perspective underpins contemporary theories of goal-directed control [14, 18, 58], which are computationally captured by the MB RL system. Unlike its MF counterpart, which relies on a cached history of past returns, an MB agent actively learns the causal structure of the world. This involves estimating two fundamental components: a transition function to predict how the environment evolves, and a reward function to map those states to potential outcomes.

The neural machinery required to build this map differs significantly from the dopamine-driven learning described in Section 2.2.1. The classic phasic dopamine response [50, 52] acts as a long-term forecaster: it computes a TD error with a horizon extending far into the future (approximating $\gamma \approx 1$), summarizing the total expected value of a trajectory [13, 29]. To build a world model, however, an agent does not need to know if an action is valuable, but simply what happens next [14]. This requires a mechanism focused on immediate, one-step consequences (effectively $\gamma = 0$). Consequently, the MB system demands a distinct signal sensitive to state transitions rather than aggregate future value to fine-tune its internal physics engine.

Where does this distinct computation reside? Some evidence suggests that MB and MF systems may not occupy separate organs, but rather map onto adjacent but structurally parallel cortico-basal ganglionic loops [14, 60, 61]. This posits that MB evaluations are supported by the same generic

dopaminergic-striatal circuit architecture that supports TD learning, simply operating over distinct cortical input representations. This anatomical similarity raises a critical question: how can the biological hardware optimized for TD caching, classically characterized by scalar reward prediction errors, implement the iterative, tree-search computations required for MB planning?

The answer likely involves a collaboration between the striatum and the medial temporal lobe [14, 62]. A longstanding assumption is that planning unfolds strictly at decision time. However, several studies suggest a more versatile architecture where planning can also occur during the encoding of information [62, 63]. When an agent discovers a new shortcut, it can infer consequences for distal actions immediately. But even with efficient encoding, the trade-off between flexibility and speed remains. Recent computational work suggests the mammalian brain resolves this by employing *depth-limited planning* [64]. When facing time pressure, humans do not simply abandon the map, but rather truncate the depth of their mental search. The agent simulates the immediate future for a few steps and then evaluates the resulting state using a learned habitual value, effectively merging the foresight of the map with the efficiency of the cache. Complementing online learning, the *Dyna* family of algorithms formalizes the role of offline planning between actions [31]. Physiologically, this process is believed to be supported by hippocampal replay [10, 14, 62]. Far from being random noise, these sequences capture the precise topological structure of the environment.

This process is reflected in hippocampal activity: during spatial tasks, the firing of place cells often sweeps ahead of the animal, suggesting a multi-step state anticipation process [10]. It is hypothesized that this hippocampal look-ahead drives the evaluation of candidate states by triggering learned reward values in the ventral striatum [56, 60, 61, 65]. This offline reactivation allows the animal to propagate new information backwards from the goal to the starting state essentially thinking through the maze while at rest. However, a critical gap remains: while we see the components, the look-ahead in the hippocampus and the valuation in the striatum, it remains unclear exactly how these signals are integrated into a larger circuit for the accumulation of evidence and the comparison of options.

2.2.3 Successor Representation: From Static Maps to Predictive Horizons

A cognitive map has long served as the dominant metaphor for hippocampal function [21], embracing the intuition that place cells encode an Euclidean representation of space. However, this static view struggles to account for the dynamic reality of neural data: place cells exhibit sensitivity to rewards, policy dependence and history effects that suggest the representation is not purely spatial, but deeply intertwined with the animal's intentions [33]. To maximize future reward, emerging theory suggests the brain needs less of a map of where the animal *is*, and more of a prediction of where it is *going*.

We have previously discussed the dichotomy between flexible, computation-heavy MB algorithms and efficient, rigid MF algorithms. Is there a computational middle ground? The solution lies in the SR [66]. This paradigm factorizes the value function into two distinct components: a predictive map of future state occupancies and a separate vector of immediate rewards. This decomposition decouples the dynamics of the environment from the value of the reward. Instead of caching the value of a state

as in MF Q-learning, the SR caches the expected future occupancy of all other states. Intuitively, the SR does not ask "How good is this state?" but rather "If I am here, where am I likely to visit next?". By storing these predictive relationships, the agent gains the flexibility of a map, allowing for rapid transfer learning if rewards change [33, 67] without the heavy computational burden of full-blown simulation.

This suggests that the hippocampus may generalize its spatial machinery to support a broader non-spatial domain: prediction. Stachenfeld et al. [18] formalized this view arguing that place cells do not encode current location coordinates, but rather the predictive horizon of the animal's current state. This theory provides a unified explanation for specific phenomena in hippocampal physiology that Euclidean models fail to predict. Compelling evidence comes from experience-dependent plasticity [18, 59, 68]: when an animal is trained to traverse a linear track repeatedly in one direction, CA1 place fields do not remain static but instead expand and shift backwards, opposite to the direction of travel. Under the SR framework, this backward skew is not an error but a mathematical necessity. As the animal's behavior becomes consistent and deterministic, the current state serves as a highly reliable predictor of upcoming states [18]. Consequently, the representation of the future projects backward into the present, allowing neurons at the start of the track to anticipate the goal. Empirical comparisons confirm that biological place fields match this SR-predicted skew far better than pure geometric models.

Beyond explaining specific firing fields, this model demonstrates highly flexible. Barry et al. [69] demonstrated that constructing the SR from a basis set of boundary vector cells (BVCs) allows the system to adapt to environmental deformations. This model maintains a coherent predictive map even when walls are stretched or barriers are inserted. Since SR separates the predictive map from the reward value, the system can rapidly adapt to new goals. Biologically, this separation implies that when the value of a goal changes, the agent need only update reward representations in the orbitofrontal cortex or striatum [70], while the hippocampal map remains intact. This multi-scale forecasting is further supported by neuroimaging evidence of a posterior-anterior gradient in the brain [25, 59]: posterior regions appear to encode immediate successors, i.e. short-term prediction, while anterior regions encode distal goals, i.e. long-term prediction, allowing the brain to solve navigation problems at different levels of abstraction simultaneously.

Finally, a major challenge has been linking the SR model to the actual activity of spiking neurons. Standard TD learning relies on a global dopamine signal to broadcast error [50], as mentioned previously. However, learning the SR requires calculating unique errors for every pair of states, a vector of errors that a single dopamine release cannot easily carry. Recent modeling provides a biologically plausible solution: the SR can be learned through spike-timing-dependent plasticity combined with presynaptic depression [71]. In this case, the prediction error is not broadcasted globally but emerges locally from the synaptic dynamics themselves. This mechanism allows the network to transition smoothly between learning modes: fast spiking sequences during replay consolidate memory using Monte Carlo-like updates, while behavioral timescale activity drives TD-like learning. By linking the computational model to neural hardware, this suggests that the cognitive map functions as a prediction of the future.

2.3 Beyond Space: Latent Representations and Generalization

Historically viewed as a Euclidean map, recent evidence suggests the hippocampus encodes any task-relevant variable. Thus, place and grid cells likely function as a general mechanism for organizing sequential relationships across both spatial and non-spatial domains [23, 72, 73].

The need of this broader framework becomes evident when we examine how the brain handles non-spatial continuous dimensions. Aronov et al. [23] trained rodents to navigate a landscape of sound frequencies rather than physical locations. Mirroring spatial representations, hippocampal and entorhinal neurons responded by forming discrete firing fields that tiled the auditory frequency axis, exhibiting the same clustering and context-dependence typically observed on a linear track. This implies that the circuit does not distinguish between movement in physical or conceptual space but simply encodes the sequential progression of relevant variables.

But how far does this abstraction extend? Neuroimaging evidence from humans suggests that this mechanism applies even to purely discrete, arbitrary relationships [6]. When participants were exposed to a sequence of objects that followed a hidden, non-spatial network structure, their hippocampal activity mirrored the topology of these *conceptual maps*. Even in the absence of explicit awareness, the brain was found to scaffold incoming information into a structured, map-like format. Organizing discrete elements into a low-dimensional manifold is essential for inference: just as spatial maps reveal shortcuts, conceptual maps uncover indirect relationships.

To provide a mechanistic basis for this generalization, recent computational models propose that the hippocampal-entorhinal circuit functions as a scaffolded heteroassociative memory [12]. In this view, the highly structured dynamics of grid cells provide a pre-existing skeleton onto which arbitrary sensory content can be added. This architecture separates the dynamics of error correction, i.e. the scaffold, from the content of the memory, avoiding the catastrophic forgetting often seen in neural networks [5, 12]. By simplifying memory into transitions between grid states, the brain efficiently balances capacity and detail, whether storing a route home or a complex novel.

This generalization capability is particularly important for goal-directed behavior in sparse-reward environments. When an agent encounters a novel task, learning from scratch is inefficient. George et al. [1, 19] propose a clone-structured cognitive graph (CSCG) that allows contexts to elastically split and merge. This flexibility enables the agent to rapidly transfer structural knowledge from previous experiences. This explains the emergence of splitter cells [74] and route-specific encoding [75]: when identical sensory inputs require different actions, the brain assigns them to distinct latent states. This effectively resolves the ambiguity, allowing for flexible and dynamic planning.

This latent state perspective reframes our understanding of neural plasticity, particularly within the CA1 region. We now observe that CA1 representations can exhibit either continuous or discontinuous remapping depending on the animal's prior experience [73]. This suggests that the circuit is not merely recording the environment but is performing bayesian state estimation using internal priors to approximate the optimal identity of the current context. In doing so, the hippocampus bridges the gap between memory and inference, transforming continuous sensory experience into a structured, navigable model.

Chapter 3

Methods

3.1 Dataset

In this study, the neural data were obtained from Jercog et al. [20] who used a high-throughput task to study goal-directed navigation under uncertainty. This protocol [20] isolates memory from sensory-motor reflexes, forcing the animal to update its internal representation to locate the reward. The behavioral box consisted of a hexadecagon (70 cm diameter) constructed from transparent vertical panels, allowing the subject visual access to distal spatial cues. Eight equidistant ports were positioned on alternating walls, equipped with infrared sensors to detect nose pokes and solenoid valves for liquid reward delivery. Control of these peripheral components and real-time tracking of the animal's position were managed via a custom Python-based interface communicating with an Arduino Mega microcontroller at a sampling rate of 100 Hz, ensuring precise temporal alignment between the animal's trajectory, port interactions and neural activity.

Following a 3-4 week pre-training period, the task was divided into two phases: a learning session and a recall session, separated by a 2-hour consolidation interval. During the 15-minute learning session, animals explored the arena to identify a single, daily-changing active port. Trial initiation required the animal to enter a *trigger zone*, an invisible, randomly placed circular area (17.5 cm diameter; 1/16th of the arena surface). Entry into this zone activated an auditory cue (pure tone; 7040-10560 Hz range), signaling the availability of water at the target port for a 6-second window. Randomizing the trigger zone location prevented the formation of motor habits, forcing the animal to rely on a flexible cognitive map rather than fixed egocentric responses.

Two hours post-learning, animals were re-introduced to the arena for the recall session to assess memory retention. To rigorously test mnemonic stability without the confound of immediate reinforcement feedback, the first 0-5 minutes of this session were conducted in extinction, where the auditory cue signaled reward availability, but no water was delivered. Following this probe period, the session reverted to the standard reinforced structure for the remainder of the 10-minute trial. This design is particularly advantageous for isolating internal simulation and goal-directed planning, as the animal must rely solely on retrieved memory traces to guide behavior in the absence of external reward signals.

To interrogate the neural substrates of this flexible behavior, calcium dynamics were recorded from the dorsal hippocampus CA1 using miniaturized head-mounted microscopes. Activity was captured from populations of up to 500 neurons per session in freely moving mice. To correct for motion artifacts inherent to active navigation, raw video frames were realigned using the NoRMCorre rigid motion correction algorithm. Individual neuronal footprints and their corresponding temporal traces were extracted using the CELLMAX algorithm, a probabilistic generative model that utilizes Expectation-Maximization to demix overlapping spatial components. The resulting $\Delta F/F_0$ traces provided a robust readout of population dynamics, enabling the simultaneous analysis of spatial coding and the non-spatial, goal-predictive signals central to this investigation.

3.2 Gridworld Simulation Suite

To rigorously test our computational hypotheses, we developed six synthetic environments that capture the essential features of the biological task. While the *in vivo* experiment operates within a continuous hexadecagon arena, our *in silico* model discretizes this space into a 10×10 squared grid, reducing the state space to a tractable size of 100 discrete states while preserving the core topological challenge.

The simulation is formalized as a Markov Decision Process (MDP) where the agent operates within a discrete action space $\mathcal{A} = \{\text{Up, Down, Left, Right}\}$. To resolve the task’s sequential dependencies, we formally define the state s_t as a layered representation:

$$s_t = i_t + \mathbb{I}(f_t) \cdot N \quad (3.1)$$

where $i_t \in \{0, \dots, N - 1\}$ is the spatial index of the grid cell (with $N = 100$), and $\mathbb{I}(f_t)$ is an indicator function representing the binary status of the trigger zone ($f_t = 1$ if activated, 0 otherwise). For deep value function approximation, this is further expanded into a concatenated feature vector $\mathbf{x}_t = [\mathbf{e}_{s_t} \oplus \mathbf{e}_g]$, incorporating the session-specific goal g to facilitate generalization across changing reward locations. State transitions are deterministic based on grid adjacency, with boundary conditions enforcing a strictly enclosed arena. To mirror the motivational constraints of the biological task, the reward function $\mathcal{R}$ relies on a sparse-positive and dense-negative structure: the agent receives a sparse reward of $+10.0$ upon reaching the designated goal state, contingent on the trigger flag being active, while incurring a step cost of -0.1 for every transition (Table C.1, Appendix C). This negative penalty serves to regularize the policy, incentivizing the discovery of the shortest path and minimal latency. Each episode is limited to a temporal horizon of $T_{\max} = 500$ steps (Table C.1, Appendix C). If the agent fails to secure the reward within this limit, the episode terminates without reward, mirroring the time-out in the behavioral box.

We introduced varying degrees of non-stationarity and structural complexity to mimic the learning and recall phases. The four corners of the grid serve as potential reward ports, while we tested several initialization state conditions (Table 3.1). To mimic the dynamic trigger zone of the biological protocol, specific environments enforce a hierarchical dependency where the agent must first visit a key state to unlock the terminal goal state. This key zone occupies 1/16th of the arena surface, matching the

spatial proportions of the experimental design by Jercog et al. [20]. The volatility of these environmental features is governed by a defined switch period $\tau = 1000$ episodes (Table C.1, Appendix C) dictating the frequency at which goal locations change. We established a hierarchy of seven environments as shown in Table 3.1 by varying goal stability, key requirements and reset conditions. This design probes cognitive capabilities ranging from simple habit formation to high-order meta-learning.

Table 3.1: Gridworld Simulation. Classification of the seven environmental conditions based on goal stability, sub-goal dynamics and reset conditions. Checkmarks indicate the active features for each environment.

Env.	Complexity	Goal Dynamics			Key Dynamics			Start Position		Cognitive Demand
		Fixed	Periodic	Random	None	Fixed	Random	Center	Continuous	
1	*	✓			✓			✓		Simple Navigation
2	**		✓		✓			✓		Contextual Memory
3	***			✓	✓			✓		Allocentric Mapping
4	****	✓				✓			✓	Hierarchical Planning
5	*****	✓					✓		✓	Sub-goal Exploration
6	*****		✓				✓		✓	Dual-Process Integration
7	*****			✓			✓		✓	Dynamic Re-planning

3.3 Reinforcement Learning

We formalize the problem of learning from interaction as a discrete-time MDP as shown in [13]. An MDP is strictly defined by the tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space and $\gamma \in [0, 1)$ represents the discount factor used to weigh the relevance of immediate versus future rewards. The dynamics of the environment are governed by the probability distribution p . At each discrete time step t , the agent observes a state $S_t \in \mathcal{S}$ and selects an action $A_t \in \mathcal{A}$. In response, the environment transitions to a new state S_{t+1} and emits a scalar reward signal $R_{t+1} \in \mathbb{R}$. This interaction generates a trajectory of experience:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots \quad (3.2)$$

The dynamics function $p : \mathcal{S} \times \mathbb{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ defines the joint probability of the next state and reward given the current state and action:

$$p(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) \quad (3.3)$$

This formulation relies on the Markov property, which asserts that the future state and reward depend solely on the current state and action, rendering the history of preceding events conditionally irrelevant.

Within this framework, the agent's behavior is defined by a policy π , a mapping from states to possible

actions. Formally, $\pi(a|s)$ is the probability that $A_t = a$ given that $S_t = s$. The objective of the agent is to maximize the expected cumulative reward over the long run. We define the return G_t as the sum of discounted future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1} \quad (3.4)$$

The recursive nature of G_t drives the efficiency of RL algorithms, allowing the decomposition of the problem into immediate rewards and discounted future value. To evaluate the quality of a policy, we employ value functions. The state-value function, denoted $v_\pi(s)$, represents the expected return when starting in state s and following policy π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \quad (3.5)$$

While $v_\pi(s)$ indicates how good it is to be in a specific state, it is often necessary to estimate the value of taking a specific action. The action-value function, denoted $q_\pi(s, a)$, is defined as the expected return starting from state s , taking action a and subsequently following policy π :

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \quad (3.6)$$

Value functions are defined by recursive consistency relationships known as the Bellman equations. These equations express the relationship between the value of a state and the values of its successor states. For the state-value function in Equation 3.5, the Bellman equation is given by:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')] \quad (3.7)$$

Intuitively, this equation states that the value of the current state is the probability-weighted sum of the immediate reward plus the discounted value of the next state. This operation is frequently referred to as a Bellman backup, as it *backs up* value information from a successor state s' to the current state s . Similarly, the Bellman equation for the action-value function in Equation 3.6 is:

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a') \right] \quad (3.8)$$

Ultimately, the goal of RL is to find an optimal policy π_* that yields a return greater than or equal to all other policies for all states, i.e. $\pi_* \geq \pi, \forall \pi$. The optimal policy shares the optimal state-value function $v_*(s) = \max_\pi v_\pi(s)$ and the optimal action-value function $q_*(s, a) = \max_\pi q_\pi(s, a)$. Under an optimal policy, the value of a state must equal the expected return for the best action from that state. This allows us to write the Bellman optimality equations, which replace the expectation over actions in Equation 3.5 with a maximization:

$$v_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \quad (3.9)$$

and for action values in Equation 3.6:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \quad (3.10)$$

Equation 3.10 underpins Q-learning. By approximating q_* , the agent derives a deterministic optimal policy through greedy action selection: $\pi_*(s) = \arg \max_a q_*(s, a)$.

3.3.1 Model-Free Algorithms

In this section, we delineate the MF RL algorithms implemented to benchmark agent performance. We organize our analysis by capability, moving from basic control methods to models designed for complex, dynamic environments. We begin with the foundational tabular approaches: Q-learning [76], SARSA and Expected SARSA [13], which establish the baseline mechanisms for off-policy and on-policy control respectively. To address the practical limitations of these basic formulations, specifically the delayed propagation of sparse rewards and estimation bias, we introduce n -step SARSA [13] and Double Q-learning [77] respectively. Subsequently, we scale these principles to high-dimensional sensory spaces using Deep Q-Networks (DQN) [78]. Finally, we detail specialized architectures tailored to the specific demands of our task: Universal Value Function Approximators (UVFA) [79] for generalizing across changing goal locations and Soft Q-learning (SQL) [45] for control under uncertainty. All hyperparameters and architectural details are summarized in Table C.2 (Appendix C).

Q-learning

To solve the RL problem without access to the underlying dynamics $p(s', r | s, a)$, we employ MF methods that rely on TD learning. A foundational approach in this domain is Q-learning, an off-policy TD control algorithm introduced by Watkins [76]. Q-learning is distinguished by its ability to approximate the optimal action-value function q_* directly from interaction samples, effectively decoupling the policy being learned, i.e. the target policy, from the policy generating the behavior, i.e. the behavior policy.

The core of the algorithm is an iterative update rule that refines the estimate of the action-value function $Q(S_t, A_t)$ at each time step. Unlike on-policy methods that estimate the value of the action actually taken, Q-learning updates the current state-action value toward the maximum predicted value of the next state, essentially assuming optimal behavior in the future. The one-step Q-learning update is defined by:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right] \quad (3.11)$$

where $\alpha \in (0, 1]$ is the step-size parameter or learning rate.

In Equation 3.11, the term in the brackets represents the TD error, the discrepancy between the current estimate $Q(S, A)$ and the improved target estimate $R + \gamma \max_a Q(S', a)$. Intuitively, the algorithm observes the immediate reward R and then *looks ahead* to the best possible value available in the next state S' . This greedy target forces the Q-function to converge toward q_* even if the agent continues to explore the environment stochastically, e.g., using an ϵ -greedy policy. This policy $\pi(a|s)$ selects the

greedy action with probability $1 - \epsilon$, while distributing the remaining probability mass ϵ uniformly among all actions to ensure exploration:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \operatorname{argmax}_{a'} Q(s, a'), \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases} \quad (3.12)$$

where $|\mathcal{A}|$ denotes the cardinality of the action space. Provided that all state-action pairs are visited infinitely often and the α decays appropriately, Equation 3.11 has been shown to converge to q_* with probability 1 [13]. The implementation of Q-learning is summarized in Algorithm 1 (Appendix E).

SARSA

While Q-learning estimates the optimal action values by decoupling the target policy from the behavior policy, an alternative approach is to learn the values of the policy actually being executed. This class of algorithms is known as on-policy control. A key algorithm in this category is SARSA [13], which offers a distinct perspective on the exploration-exploitation trade-off.

In contrast to Equation 3.11, which updates a state-action pair based on the maximum possible value of the next state, SARSA updates its estimates based on the action actually selected by the current policy in the next state. This distinction is significant: while Q-learning assumes optimal future behavior, SARSA accounts for the specific trajectory taken by the agent, including exploratory moves. The update rule for SARSA is defined as:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)] \quad (3.13)$$

The name "SARSA" is derived directly from the quintuple of events (S, A, R, S', A') required to compute the update in Equation 3.13. Consequently, this affects how the agent navigates areas with potential high penalties. For example, in an environment with a cliff where falling results in a large negative reward, Q-learning will learn the optimal path right along the edge, assuming it will not slip, whereas SARSA, recognizing that its ϵ -greedy policy (Equation 3.12) might cause it to fall occasionally, will prefer a safer path further from the edge. Thus, SARSA often yields better online performance during the learning phase, but only converges to the optimal policy if ϵ is decayed to zero over time.

Algorithm 2 (Appendix E) outlines this on-policy procedure: the next action A' must be selected before the update step to calculate the target value, tying the update to the current behavior policy.

Expected SARSA

To balance the stability of on-policy learning with the optimality of off-policy updates, we employ Expected SARSA [80]. While SARSA estimates the value of the current policy by sampling specific action trajectories, thereby introducing variance dictated by the exploration strategy, Expected SARSA marginalizes over the action space to compute the deterministic expected value of the next state. The update rule is defined as:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \sum_a \pi(a|S') Q(S', a) - Q(S, A) \right] \quad (3.14)$$

By replacing the sampled transition with this expectation, the algorithm effectively eliminates the variance arising from policy stochasticity, e.g., ϵ -greedy noise, limiting the source of variance to the dynamics of the environment itself. This reduction in variance permits higher learning rates, accelerating convergence toward the optimal policy without the instability characteristic of Q-learning [81].

For Goal 2 (Section 1.3), Expected SARSA serves as a generalization of Q-learning. Mathematically, Q-learning is simply the special case where the expectation is calculated over a deterministic greedy policy. This flexibility allows Expected SARSA to strictly mimic Q-learning's off-policy maximization when needed, while retaining the capacity to evaluate exploratory policies. This provides a mechanism to estimate values even in the presence of the aliased sensory feedback inherent to our task.

n -Step SARSA

While single-step TD methods like Q-learning and SARSA constitute the fundamental building blocks of RL, they face inherent limitations in complex environments, particularly those characterized by sparse reward signals or large state spaces. In their basic tabular form, these algorithms rely on *bootstrapping*, i.e. estimating the value of a state based solely on the estimated value of the immediate successor. Therefore, reward information propagates backward only one step per episode. In our spatial navigation task, where an agent traverses long corridors without feedback, single-step propagation can become inefficient, resulting in slow learning rates.

To address this, we employ n -step TD learning [29], a method that balances local one-step updates with global Monte Carlo methods. Whereas Monte Carlo methods wait for the full return G_t and one-step TD updates immediately with R_{t+1} , n -step methods accumulate rewards over n steps before bootstrapping. This allows the agent to capture longer-term dependencies and propagate reward signals more efficiently along the trajectory. We can define the n -step return $G_{t:t+n}$ as the sum of the first n discounted rewards plus the discounted estimated value of the state reached at time $t + n$. For an on-policy method like SARSA, this is given by:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n}) \quad (3.15)$$

where $0 \leq t < T - n$. If the episode ends before n steps, i.e., $t + n \geq T$, the summation effectively becomes the full Monte Carlo return. The update rule for n -step SARSA shifts the estimate $Q(S_t, A_t)$ toward this n -step return:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_{t:t+n} - Q(S_t, A_t)] \quad (3.16)$$

We hypothesize that n -step methods offer a behavioral advantage in aliased or noisy environments. By using a sequence of n observations, the agent smooths out step-by-step noise, resulting in more stable trajectories. In spatial navigation, this enables the agent to learn path *chunks* that reliably lead

to landmarks, accelerating policy discovery compared to single-step methods. The implementation in Algorithm 4 (Appendix E) requires a temporary memory buffer to store the last n states and rewards. The update for time step t is performed at time $t + n$, introducing a delay equal to n steps.

Double Q-learning

While Q-learning provides a simple off-policy control mechanism, it suffers from a systematic bias inherent to its maximization operator. In Q-learning, the update rule in Equation 3.11 uses the same estimator both to select the greedy action, by maximizing the value, and to evaluate that action's value. Hence, this creates a positive bias because, for any set of random variables, the expected value of the maximum is greater than or equal to the maximum of the expected values:

$$\mathbb{E} \left[\max_a \hat{Q}(s', a) \right] \geq \max_a \mathbb{E} \left[\hat{Q}(s', a) \right] \quad (3.17)$$

In stochastic environments this maximization bias leads to the overestimation of action values. The agent may favor sub-optimal actions simply because their values have high variance, interpreting statistical noise as potential reward. This effect occurs even in MDPs with deterministic rewards, as long as state transitions are stochastic. This is particularly relevant to our navigation task, where sensory aliasing introduces transition uncertainty.

To rectify this, we implement Double Q-learning by van Hasselt [77], an algorithm that decouples the selection of the optimal action from the evaluation of its value. The core principle involves maintaining two independent value estimates, Q^A and Q^B , initialized arbitrarily. At each time step, we update one estimate Q^A using the other Q^B to calculate the value of the next state. The update rule for Q^A is:

$$Q^A(S, a) \leftarrow Q^A(S, a) + \alpha [R + \gamma Q^B(S', a^*) - Q^A(S, a)] \quad (3.18)$$

where the action a^* is selected greedily with respect to the target function Q^A :

$$a^* = \operatorname{argmax}_a Q^A(S', a)$$

Note that while a^* maximizes Q^A , its value is estimated using $Q^B(s', a^*)$. Since Q^B is updated using a separate set of experience samples, it provides an unbiased estimate of the value of a^* . The analysis in [77] demonstrates that while the single estimator in Q-learning is positively biased, this double estimator is unbiased or slightly negatively biased leading to underestimation. In spatial navigation, this conservative estimation ensures the agent constructs trajectories based on reliable signals rather than random noise.

Despite doubling the number of value functions, the algorithm remains data-efficient [77] because it uses every experience tuple (s, a, r, s') to update one of the two estimators. Convergence to the optimal policy is guaranteed under the Robbins-Monro stochastic approximation conditions [82], provided the learning rate α decays appropriately respecting $\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$. While Q-learning can exhibit exponential convergence time in pathological cases, Double Q-learning frequently demonstrates

superior polynomial time convergence in stochastic settings by dampening the oscillations caused by overestimation [77]. The implementation is detailed in Algorithm 5 (Appendix E).

Deep Q-Networks

The tabular methods described previously are fundamentally limited by the *curse of dimensionality*. In biological environments with complex inputs like vision and other egocentric sensations, the state space is too vast for a table. Single snapshots of sensory data often fail to capture the full system dynamics. For instance, a static image of a maze lacks velocity or direction, which makes the task partially observed. Consequently, the agent faces ambiguity: different locations in the environment often look exactly the same, making them indistinguishable from immediate observations alone.

To overcome these limitations and benchmark performance in complex biological settings as stated in Goal 1 (Section 1.3), we implement the DQN architecture [78]. Rather than mapping discrete states to values, DQN approximates the optimal action-value function, $Q^*(s, a)$, using a non-linear neural network with weights θ , such that $Q(s, a; \theta) \approx Q^*(s, a)$. To solve perceptual aliasing, we define the state not as the immediate observation x_t , but as the history of interactions: a sequence of actions and observations $s_t = x_1, a_1, x_2, \dots, x_t$. This transformation converts the partially observed problem into a finite MDP where each sequence is a distinct state, allowing the application of standard RL principles.

The network is trained to minimize the TD error. Unlike supervised learning where targets are fixed, RL targets are non-stationary, depending on the network’s evolving weights. The loss function at iteration i , denoted as $L_i(\theta_i)$, is defined as the expected squared error between the network’s prediction and a target value derived from the Bellman equation:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[(y_i - Q(s, a; \theta_i))^2 \right] \quad (3.19)$$

where $U(D)$ represents a uniform distribution over transitions stored in an experience replay memory D . The target y_i represents the best estimated return, computed using weights θ_{i-1} from the previous iteration:

$$y_i = r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})$$

Differentiating the loss function with respect to the weights yields the gradient update rule used for stochastic gradient descent:

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right] \quad (3.20)$$

Instead of discarding immediate data, the agent stores its experiences in a memory buffer and later replays random snippets to learn. This experience replay mechanism breaks the temporal correlations in the training data, stabilizing the non-linear function approximation. By learning directly from high-dimensional inputs, DQN allows us to model how biological agents might extract optimal policies from complex, aliased sensory streams without pre-defined state representations.

Universal Value Function Approximators

Although DQN is able to learn from complex states that integrate spatial and non-spatial domains as depicted in Section 3.2, it relies on a static reward function limiting the agent to a single objective. This limitation proves important in our task, where the goal location is relocated every session [20]. A standard DQN agent would require computationally expensive retraining for every new goal configuration, rendering it inefficient for such dynamic environments. To address this, we implement UVFA [79], which generalizes value functions by separating environment dynamics from goal representations. The main innovation is to represent a large set of optimal value functions within a single, unified function approximator that estimates the long-term reward conditioned on both the current state $s \in \mathcal{S}$ and a specific goal $g \in \mathcal{G}$.

UVFA extends the standard estimate $Q^*(s, a)$ by approximating the optimal goal-conditioned value $Q(s, a, g; \theta) \approx Q_g^*(s, a)$. This formulation allows the agent to cache knowledge about the environment’s structure in a way that is transferrable across different objectives. By learning the structure of the joint space $\mathcal{S} \times \mathcal{G}$, the agent can immediately assess the utility of actions for a goal it may have never explicitly trained on, assuming that the goal shares structural similarities with previously visited states. While Schaul et al. [79] originally proposed a *two-stream* architecture involving low-rank matrix factorization to combine state and goal features, we implement the concatenated architecture to facilitate stable end-to-end training without complex factorization. In this approach, the state vector s and the goal vector g are concatenated to form a joint input vector $h = [s, g]$, which is passed through a non-linear multi-layer perceptron, effectively treating the goal as an additional context input. Indeed, training is performed end-to-end by minimizing the mean-squared error between the predicted values and the Bellman targets, generalizing the loss function to include goals:

$$L(\theta) = \mathbb{E}_{s,a,g,r,s'} \left[\left(r + \gamma \max_{a'} Q(s', a', g; \theta^-) - Q(s, a, g; \theta) \right)^2 \right]. \quad (3.21)$$

This architecture attempts to overcome the challenges of reward sparsity and dynamic goals inherent to our dataset [20]. By enabling the agent to generalize over goals, we satisfy the requirements for flexible control policies outlined in Goal 2 (Section 1.3), allowing the agent to navigate to changing daily targets without retraining. The implementation is summarized in Algorithm 7 (Appendix E).

Soft Q-learning

The algorithms discussed so far, both DQN and UVFA, operate on a conventional notion of optimality: they seek a deterministic policy that maximizes the expected cumulative reward. While theoretically optimal in fully observable, static environments, such *hard* maximization is often brittle in the face of uncertainty. In contrast, biological agents frequently exhibit stochastic behavior, maintaining a portfolio of viable strategies rather than a single deterministic policy. This stochasticity is essential for robustness, particularly in environments where sensory aliasing renders the state partially unobservable. To capture this biological flexibility, we adopt the framework of maximum entropy RL, specifically adapting the SQL approach of Haarnoja et al. [45]. In this method, the objective function is augmented to maximize not

just the expected return, but also the entropy of the policy $\mathcal{H}(\pi(\cdot|s_t))$:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))], \quad (3.22)$$

where α is a temperature parameter determining the relative importance of entropy (i.e. exploration) versus reward (i.e. exploitation). Drawing from the connection between optimal control and probabilistic inference [45], we formulate the stochastic policy as a conditional energy-based model, where the optimal policy follows a Boltzmann distribution over the soft action-value function: $\pi^*(a|s) \propto \exp(\frac{1}{\alpha} Q_{soft}(s, a))$.

Unlike the continuous control formulation proposed in [45], which typically requires a separate actor network to approximate the intractable partition function over an infinite action space, our discrete action space allows for a significant architectural simplification. Because we can enumerate all possible actions, the partition function is analytically tractable, allowing us to define the soft value of a state $V_{soft}(s)$ exactly as the log-sum-exponent of the Q-values:

$$V_{soft}(s) = \alpha \log \left(\sum_{a'} \exp \left(\frac{Q_{soft}(s, a')}{\alpha} \right) \right). \quad (3.23)$$

This derivation reveals a critical distinction from the standard DQN update: whereas DQN estimates the value of the best action given by $\max_a Q$, SQL estimates the aggregated value of all viable actions. Intuitively, this allows the agent to recognize states that offer multiple good paths as distinct from states with a single high-risk peak. The training objective minimizes the Bellman error using this soft value estimate for the bootstrap target:

$$\mathcal{L}(\theta) = \mathbb{E}_{s, a, r, s'} \left[(Q(s, a; \theta) - (r + \gamma V_{soft}(s'; \theta^-)))^2 \right]. \quad (3.24)$$

By optimizing this objective, the agent naturally balances exploration and exploitation. The entropy term prevents the policy from converging prematurely to deterministic behavior, ensuring the agent maintains a soft optimality capable of adapting to the sparse rewards and dynamic goal relocations characteristic of our behavioral task. The implementation is detailed in Algorithm 8.

3.3.2 Model-Based Algorithms

The MF algorithms described in the previous section rely exclusively on direct experience to estimate value functions. While effective, this approach is often sample-inefficient [33] as the agent must physically revisit states to propagate reward information. To address this, we implement MB RL, specifically the Dyna-Q architecture [31]. Dyna-Q integrates online planning with direct learning, dividing resources between two processes: model learning, which refines the internal representation, and planning, which updates the policy without real-world interaction. Table C.3 (Appendix C) summarizes all model hyperparameters and architectural details.

Dyna-Q

The core of the Dyna-Q architecture [31] is the maintenance of an internal model $M(s, a) \rightarrow (r, s')$, which approximates the transition dynamics $p(s', r | s, a)$ by memorizing the most recently observed outcome for each state-action pair. For every real experience (S, A, R, S') , the agent performs a direct Q-learning update (Equation 3.11) and records the transition in its internal model $M(S, A) \leftarrow (R, S')$. Subsequently, the agent enters the planning phase, executing N steps of *hallucinated* experience where $N = 10$. In each planning step, the agent randomly samples a previously visited state s and an action a previously taken in that state, queries the model for the predicted outcome and applies Equation 3.11 to this simulated transition:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[M_R(S, A) + \gamma \max_{a'} Q(M_S(S, A), a') - Q(S, A) \right] \quad (3.25)$$

This mechanism allows reward signals to propagate through the state space significantly faster than direct experience alone would permit, addressing the challenge of sparse rewards.

The efficacy of this planning process is heavily contingent on the exploration strategy employed. We investigate two distinct action selection mechanisms. The standard Dyna-Q implementation uses an ϵ -greedy policy, which balances exploitation with undirected random exploration: $\pi(a|s) = 1 - \epsilon + \epsilon/|\mathcal{A}|$ for the greedy action and $\epsilon/|\mathcal{A}|$ otherwise. However, to foster more intelligent exploration, particularly in non-stationary environments where the model must adapt to changing goals, we also implement another Dyna-Q variant [83]. This method treats action selection as a multi-armed bandit problem, using an Upper Confidence Bound (UCB) to favor actions with high uncertainty. The selection rule is given by:

$$A_t = \operatorname{argmax}_a \left[Q(S_t, a) + c \sqrt{\frac{2 \ln \sum_{a'} N(S_t, a')}{N(S_t, a)}} \right] \quad (3.26)$$

where $N(s, a)$ denotes the visit count for the state-action pair and c is a confidence parameter (Table C.3, Appendix C). This *directed exploration* grounds planning in a uniformly verified model, reducing errors from incorrect dynamics.

3.3.3 Successor Representation Algorithm

The previous algorithms present a fundamental dichotomy in RL control. MF methods are computationally efficient at decision-time, but brittle to changes in reward structure whereas MB methods offer flexibility but incur a high computational cost for planning [33]. In spatial navigation and hippocampal function, this binary classification is increasingly viewed as insufficient. Stachenfeld et al. [18] suggest the hippocampus acts as a predictive map via the SR, effectively sitting between these two extremes. The SR architecture proposed in [66] factorizes the value function into two distinct components: a predictive representation of the environment's dynamics and a separate representation of the reward structure. This factorization allows an agent to rapidly adapt to changes in goal location without the need for extensive retraining or computationally expensive trajectory simulation.

The SR emerges naturally from the Equation 3.5. The value of a state $V(s)$ is defined as the expected sum of discounted future rewards. Assuming state-dependent rewards, we can decompose the value function into the inner product of a reward vector R and the SR M :

$$V(s) = \sum_{s'} M(s, s') R(s'). \quad (3.27)$$

where $M(s, s')$ encodes the expected discounted future occupancy of state s' along a trajectory initiated in state s :

$$M(s, s') = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{I}(S_t = s') \mid S_0 = s \right], \quad (3.28)$$

where $\mathbb{I}(S_t = s') = 1$ if the agent is in state s' at time t and 0 otherwise. While $R(s')$ maps how good a state is, $M(s, s')$ maps how likely the agent is to visit s' from s . In our implementation, we extend this framework to the control setting by defining an action-dependent SR tensor $M(s, a, s')$, representing the future occupancy given that action a is taken in state s . The action-value function $Q(s, a)$ is then computed as the dot product between this predictive map and a learned reward weight vector w :

$$Q(s, a) = \sum_{s'} M(s, a, s') \cdot w(s'). \quad (3.29)$$

Like Q-learning, M is learned incrementally using TD errors, avoiding the need for a full generative model. When the agent transitions from state s to s' via action a , the SR is updated to satisfy the recursive Bellman property: current occupancy plus discounted future occupancy. The update rule is:

$$M(S, A, :) \leftarrow M(S, A, :) + \alpha [\mathbf{1}_S + \gamma M(S', A', :) - M(S, A, :)] \quad (3.30)$$

where $\mathbf{1}_s$ is a one-hot vector representing the current state S , and A' is the greedy action in the next state. Simultaneously, the reward weights w are updated via a simple supervised error signal based on the immediate reward observed: $w(S') \leftarrow w(S') + \alpha (R - w(S'))$. This factorization enables the agent to essentially propagate value information: if the goal location changes, the agent only needs to update w locally at the new goal state. The stable map M instantly propagates this new value to distant states via the dot product (Equation 3.30), handling dynamic goals more efficiently than MF baselines. The implementation is in Algorithm 10 (Appendix E).

3.3.4 Performance Metrics

Final Average Reward

This metric measures post-convergence optimality by averaging the final W rewards, excluding exploration noise to assess the true quality of the learned policy.

$$\bar{R}_{final} = \frac{1}{W} \sum_{i=E-W}^E R_i \quad (3.31)$$

where R_i is the reward in episode i , E is the total episodes and $W = 100$ is the window size.

Total Cumulative Reward

Serving as a proxy for total regret, this metric sums rewards across the entire training period. It differentiates fast learners from slow ones by capturing the total cost of acquisition, even if final performance is identical.

$$R_{total} = \sum_{i=1}^E R_i \quad (3.32)$$

Convergence Efficiency

This metric quantifies the temporal speed of learning by identifying exactly when the agent solved the task. We define convergence as the episode index where the agent consistently achieves 90% of the maximum possible reward R_{max} . This is decisive for comparing algorithms in resource-constrained environments where training time is limited.

$$E_{conv} = \min\{e \mid \max_{i \leq e} (R_i) \geq 0.9 \cdot R_{max}\} \quad (3.33)$$

Optimality Gap

This metric assesses the spatial quality of the agent's navigation. Using the Manhattan distance to define the theoretical shortest path, it calculates the ratio against the number of steps the agent actually took to the goal. A value of 1.0 indicates a perfect trajectory, while values greater than 1.0 reveal sub-optimal behaviors such as unnecessary detours or loops.

$$\eta = \frac{1}{E} \sum_{i=1}^E \frac{S_i}{D_M(start, goal)} \quad (3.34)$$

where S_i is the steps taken in episode i and D_M is the optimal distance.

Stability

Stability measures the consistency of the agent's behavior after convergence. It penalizes agents that oscillate between good and bad policies during the asymptotic phase. In biological contexts, low variance is often preferred over high-risk maximization. This metric identifies agents that maintain a steady policy. We quantify this using the standard deviation of rewards over the final $W = 100$ episodes:

$$\sigma = \sqrt{\frac{1}{W} \sum_{i=E-W}^E (R_i - \bar{R}_{final})^2} \quad (3.35)$$

Recovery Time

In dynamic environments where the goal location shifts, this metric measures the agent's plasticity. It counts the number of episodes required for the agent to obtain a positive reward immediately following a change in context. This serves as the key test to distinguish MB planning from MF habits:

$$\Delta E_{rec} = \frac{1}{K} \sum_{k=1}^K (e_{fr}^{(k)} - e_{sw}^{(k)}) \quad (3.36)$$

where $e_{sw}^{(k)}$ is the episode index when the k -th goal switch occurred, $e_{fr}^{(k)}$ is the first subsequent episode where the agent found the new goal and K is the total number of context switches. In our implementation, one switch occurs every 1,000 episodes.

Chapter 4

Results

4.1 Expected Results

To interpret the benchmarking results, we analyze the mathematical structure of each algorithm's update rule. We attribute these performance differences to the fundamental bias-variance trade-off in MF tabular agents. Furthermore, performance is dictated by the specific structural assumptions each method imposes on the value function, essentially, how it chooses to link related states together. This structure is important, as it determines the actual utility of the state representation for solving the task in each specific environment.

4.1.1 Maximization Bias and Stability: The Jensen Gap

Stochastic transitions $P(s'|s, a)$ induce zero-mean noise $Y_{s,a} \sim \mathcal{N}(0, \sigma_Q^2)$ in the value estimates $\hat{Q}(s, a) = Q^*(s, a) + Y_{s,a}$. Standard Q-Learning updates based on the target $y = r + \gamma \max_a \hat{Q}(s', a)$. By Jensen's Inequality, the convexity of the maximization operator transforms this symmetric noise into a positive bias:

$$\mathbb{E} \left[\max_a \hat{Q}(s, a) \right] = \mathbb{E} \left[\max_a (Q^*(s, a) + Y_{s,a}) \right] \geq \max_a Q^*(s, a) \quad (4.1)$$

This inequality creates a systematic overestimation error that scales with the estimator variance σ_Q [84]. In our task, where the randomized trigger zone locations described in [20] induce high initial variance, we hypothesize this will destabilize Q-Learning. Double Q-Learning rectifies this by decoupling selection from evaluation using two independent estimators (Q^A, Q^B) . By selecting actions with one estimator and evaluating them with the other, the target becomes unbiased $\mathbb{E}[y_d] = Q^*$, effectively eliminating the maximization bias [77] and enhancing stability.

4.1.2 Variance Reduction in On-Policy Control: Expected SARSA

Under a stochastic ϵ -greedy policy (Equation 3.12), the standard SARSA target Y_{SARSA} fluctuates due to both environmental dynamics R, S' and random action selection A' :

$$Y_{SARSA} = R + \gamma Q(S', A'), \quad A' \sim \pi(\cdot | S')$$

Expected SARSA eliminates the noise from A' by explicitly marginalizing over the policy. The target Y_{EXP} depends solely on environmental factors:

$$Y_{EXP} = R + \gamma \sum_{a'} \pi(a' | S') Q(S', a') = \mathbb{E}_{\pi}[Y_{SARSA} | R, S']$$

Using the Law of Total Variance, we decompose the variance of the SARSA target into the variance of the expected target plus the variance due to action selection:

$$Var(Y_{SARSA}) = Var(Y_{EXP}) + \underbrace{\mathbb{E}[Var(Y_{SARSA} | R, S')]}_{\text{Policy Noise} \geq 0}$$

Since the policy noise term is non-negative, Expected SARSA guarantees a reduction in variance [81]:

$$Var(Y_{EXP}) \leq Var(Y_{SARSA})$$

Intuitively, this acts as a stabilizing filter. By averaging over all possible actions rather than sampling one, the agent avoids downgrading valid paths due to random exploration errors, e.g. accidentally stepping into a wall, maintaining a coherent value map even during high-exploration phases.

4.1.3 Balancing Learning Speed and Noise: n -Step SARSA

The convergence efficiency in sparse-reward environments is constrained by the rate at which reward information propagates from the goal back to the start state. 1-step methods propagate this signal slowly, scaling quadratically with the grid size. n -Step SARSA accelerates this by using the truncated Monte Carlo return $G_{t:t+n}$, which we can re-write from Equation 3.15 as:

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q(S_{t+n}, A_{t+n})$$

The performance of this estimator is governed by the decomposition of its mean squared error [13]. We can express the error explicitly in terms of the horizon n :

$$MSE(G_{t:t+n}) \approx (\gamma^n [\mathbb{E}[Q(S_{t+n})] - Q^*(S_{t+n})])^2 + Var\left(\sum_{k=0}^{n-1} \gamma^k R_{t+k+1}\right) \quad (4.2)$$

The first term represents the bias: it measures how much the current, imperfect value estimate $Q(S_{t+n})$ corrupts the target. Increasing n exponentially reduces the weighting factor γ^n , dampening the

impact of inaccurate initial estimates and improving convergence. On the other hand, the second term represents the variance: it is the sum of stochastic fluctuations over n transitions. In our environment, where each step involves probabilistic state transitions, this noise accumulates additively. We attribute the stability collapse observed at $n = 5$ (Table C.2, Appendix C) to this phenomenon. At this horizon, the accumulated noise term likely overwhelms the reward signal during early training, preventing the formation of a stable gradient.

4.1.4 Soft Optimality Improves Exploration: Soft-Q Learning

Greedy policies $\pi(s) = \arg \max Q(s, a)$ suffer from a *winner-take-all* dynamic that collapses the policy distribution onto a single mode [45]. This proves detrimental in our hierarchical task: if the trigger zone path is initially estimated as sub-optimal, a greedy agent assigns it zero probability, effectively ceasing exploration. Hence, SQL resolves this by reformulating control as probabilistic inference [44]. The optimal value function is defined not by the hard maximum, but by the *LogSumExp* operator as shown in Equation 3.23. The α parameter tunes the entropy of the policy. The critical advantage of this formulation is visible in its gradient. Unlike the hard maximum, which has a gradient of 0 for all non-optimal actions, the gradient of the soft value function is non-zero with respect to all action values:

$$\frac{\partial V_{soft}(s)}{\partial Q(s, a)} = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)} = \pi_{soft}(a|s)$$

This property ensures that learning signals propagate even through currently sub-optimal actions. We expect this mechanism to significantly reduce the number of steps needed to find the trigger zone each episode. The entropy term $\alpha \mathcal{H}(\pi)$ in the objective function (Equation 3.22) acts as an intrinsic exploration bonus, preventing the agent from becoming trapped in local optima near the goal, when the trigger zone has not been found, and ensuring sustained exploration of the trigger zone path even when it yields lower immediate returns.

4.1.5 Deep Representation and Goal-Conditioned Generalization

To scale our analysis to high-dimensional state spaces while preserving behavioral interpretability, we contrast the direct value approximation of DQN with the structured factorization of UVFA. Both architectures approximate $Q(s, a; \theta)$ via gradient descent, yet their specific handling of goal states drives the differences in adaptability and recovery speed.

DQN combines the state and goal into a single input, effectively embedding the goal location directly into its weights. When the goal shifts ($g \rightarrow g'$), the previous mapping becomes invalid. The agent must essentially "unlearn" the old goal before learning the new one, causing slow recovery times. On the other hand, UVFA resolves this rigidity by imposing a manifold constraint via a two-stream architecture depicted in [79]. Instead of a *black-box* mapping, we enforce a geometric factorization $Q(s, a, g) \approx \phi(s)^T \cdot \psi(g)$, where the value function arises from the inner product of two independent embeddings [79]. Here, $\phi(s)$ encodes the structural dynamics, such as adjacency and walls layout, while $\psi(g)$ encodes

the reward location. This factorization structurally decouples the agent’s cognitive map from its current desires. A goal shift corresponds to a simple rotation of the $\psi(g)$ vector within the fixed eigenspace of $\phi(s)$. We argue that this architecture will enable zero-shot transfer: the agent reuses its learned environmental model $\phi(s)$ to navigate to novel goals immediately, yielding the near-zero recovery latency required to mimic the latent learning capabilities of biological agents. This architectural decoupling mirrors the mechanics of the SR, where value is factorized into predictive occupancy and reward vectors as shown in Equation 3.27. This shared computational motive suggests that explicitly separating map from goal is the critical inductive bias that enables the efficiency spatial navigation in rodents [18].

4.1.6 Model-Based Planning and Uncertainty-Driven Exploration

In sparse-reward regimes such as the initial search for the trigger zone, Q-Learning suffers from high sample complexity because value information propagates strictly backward from the reward source, one state per physical step. To address this, we employ the Dyna-Q architecture [30], which integrates a learned world model $M(s, a) \rightarrow (r, s')$ to decouple learning from physical actuation. By performing K planning updates for every physical step using simulated experiences $(\tilde{s}, \tilde{a}, \tilde{r}, \tilde{s}')$, the agent effectively *hallucinates* trajectories to propagate value information using the update rule in Equation 3.25.

Functionally analogous to hippocampal replay [15], this mechanism should improve convergence and reduce path lengths. It enables the agent to discover the optimal path via background planning, removing the need to physically retrace the arena. However, the efficacy of this planning relies heavily on the exploration policy. However, the efficacy of this planning depends heavily on the exploration policy. Standard Dyna-Q uses ϵ -greedy, which relies on random action selection. We predict this will yield high variance, as the probability of stumbling upon distant sub-goals by chance is low. On the other hand, Dyna-Q with UCB replaces stochasticity with deterministic optimism. By selecting actions that maximize an UCB as shown in Equation 3.26, where the agent assigns an intrinsic value to the unknown. The bonus term $\sqrt{\frac{2 \ln \sum_{a'} N(S_t, a')}{N(S_t, a)}}$ creates a transient reward gradient that actively pulls the agent toward unvisited states. We predict that this exploration bonus will enable Dyna-Q with UCB to surpass ϵ -greedy variants, mimicking the directed search strategies of biological agents in novel environments.

4.1.7 Cognitive Mapping and Latent Learning: Successor Representation

It has been proposed that the hippocampus mitigates the trade-off between flexible planning and efficient execution by encoding the SR [18], a predictive map that linearly factorizes value into environmental dynamics and reward utility. This structure decouples the stable cognitive map from transient goals. Crucially, the policy relies on the inner product of a learned predictive occupancy map M and a reward vector w : $Q(s, a) = M(s, a) \cdot w$. This uncoupling allows us to derive the mechanism of latent learning. When the goal location shifts $w \rightarrow w'$, the value update at any distant state s is not recursive

but instantaneous, as shown previously in Equation 3.29:

$$Q(s, a) = \sum_{s'} \underbrace{M(s, a, s')}_{\text{Invariant Map}} \cdot \underbrace{w(s')}_{\text{Goal}}$$

Here, the $M(s, a, s')$ encodes the spatial structure of the environment. Since the maze layout is constant, this predictive map remains valid across different tasks specifically to shifts in goal locations. Consequently, the agent leverages this pre-learned topology to immediately infer the new optimal path without re-exploration. We hypothesize that this vector-matrix multiplication is the computational driver of near-zero recovery latency, providing a model for the zero-shot flexibility of biological goal-directed behavior in spatial navigation tasks.

4.2 Preliminary Results

We focus our analysis on Environments 5 and 7 (Table 3.1), as their stochastic protocols mirror the behavioral tasks in the rodent study by Jercog et al. [20]. In this regime, agents must first locate a random 2×2 trigger zone to unlock the active goal as described in Section 3.1. This goal location shifts randomly every 1,000 episodes, a duration specifically selected to ensure qualitative convergence before relocation. Our evaluation proceeds in two stages. First, we validate the RL agents against the performance metrics in Section 3.3.4 to identify the best-performing models as proposed in Goal 1 (Section 1.3). Subsequently, we compare these agents using behavioral metrics to align with biological tracking data.

Analysis of the MF tabular agents reveals significant trade-offs between stability and recovery times. As illustrated in the stability panel (Figure 4.1, *Bottom-Mid*), Double Q-learning (beige bar) emerged as the most robust architecture. It exhibits the lowest value among the tabular methods, indicating minimal variance in post-convergence rewards compared to the higher volatility observed in Q-Learning and SARSA. This performance supports the Jensen Gap hypothesis described in Section 4.1.1: by decoupling action selection from evaluation, the algorithm mitigates maximization bias, allowing the agent to converge on safer policies. In contrast, n -Step SARSA (green bar) deviated significantly from this trend. Looking at the recovery time panel (*Bottom-Right*), it achieved a significant reduction in adaptation lag, performing comparably to planning agents. However, this came at the cost of precision: the stability plot (*Bottom-Mid*) shows that propagating reward signals over longer horizons introduced high variance due to the accumulation of transition noise. Regarding Expected SARSA (orange bar), we hypothesized that its update rule would act as a stabilizing filter (Section 4.1.2) in the stability (*Bottom-Mid*). Intuitively, while SARSA agent (light blue bar) might erroneously downgrade a correct path's value due to a random sub-optimal next step, such as hitting a wall, Expected SARSA averages out these mistakes. We predicted that this theoretical immunity to exploration noise would allow the agent to maintain a coherent value map. However, the data contradicts this expectation. As seen in the stability (*Bottom-Mid*), the Expected SARSA bar (orange bar) shows no significant reduction compared to SARSA (light blue bar). This suggests that this specific variance-reduction mechanism is not an

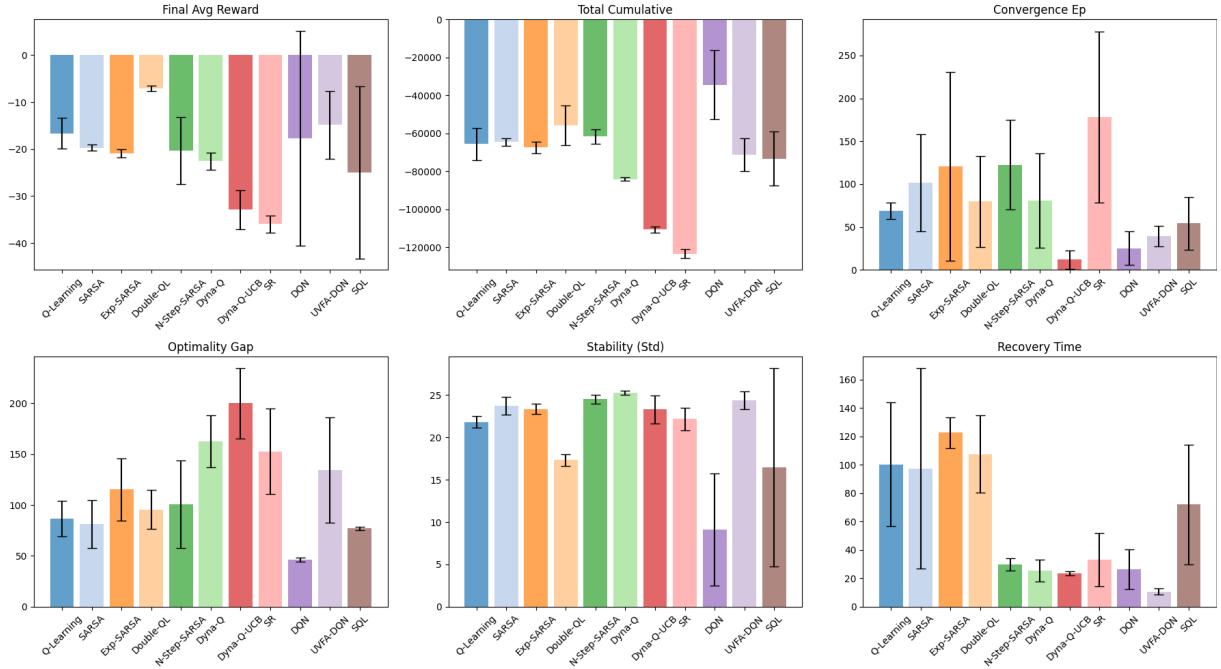


Figure 4.1: Performance Metrics in Environment 7. Quantitative analysis of MF, MB and SR agents over 4,000 episodes, with random goal relocations every 1,000 episodes. (*Top-Left*) Final Average Reward: Measures asymptotic proficiency by averaging rewards over the final 100 episodes; higher is better. (*Top-Mid*) Total Cumulative Reward: Proxy for total regret, distinguishing fast learners from slow learners based on the total cost of the learning process. (*Top-Right*) Convergence Ep: The first episode index where the agent consistently achieves 90% of maximum reward; lower indicates faster learning. (*Bottom-Left*) Optimality Gap: Ratio of steps taken versus the optimal Manhattan distance; values > 1.0 indicate inefficient loops or detours. (*Bottom-Mid*) Stability: Standard deviation of rewards at the last 100 episodes; lower values indicate a consistent policy. (*Bottom-Right*) Recovery Time: Mean episodes required to secure a positive reward immediately following a goal switch, measuring plasticity. Error bars represent the standard error of the mean across $n = 3$ independent trials. Note the large variance in Recovery Time and Convergence for SR and DQN, indicating that performance differences in these specific metrics may lack statistical significance due to the high stochasticity of the trigger mechanism.

essential feature for solving this task structure. Finally, as seen in the path efficiency results (*Bottom-Left*), MF tabular agents occupy an intermediate position. While SQL (brown bar) and DQN agents (purple bar) achieve superior path efficiency compared to MB approaches, this optimality highlights a potential divergence from biological reality. Real rodents often exhibit inefficient paths necessitated by the search for the stochastic trigger zone. This implies that optimality gap is a poor predictor of natural animal behavior in this task.

Turning to the Deep RL architectures in Figure 4.1, the recovery time (*Bottom-Right*) illustrates a shift in plasticity. The DQN (purple bar) and UVFA-DQN (light purple bar) agents exhibit fast recovery, locating the new goal in roughly 30 episodes. This performance is better than the tabular MF tabular agents, which required significantly longer adaptation periods. These results highlight that explicitly encoding non-spatial context, such as goal identity and trigger status, is a key driver for rapid adaptation. We attribute this to contextual generalization: rather than relearning a spatial map cell-by-cell, the network uses the goal and trigger zone input to instantly re-map values across the field, mirroring the hippocampal ability to switch cognitive maps.

However, while the UVFA agent (light purple bar) matches the rapid recovery of DQN (purple bar), it exhibits significantly higher variance after convergence (*Bottom-Mid*). This suggests that the agent's

two-stream architecture suffers from a trade-off: the same mechanism that allows it to instantly adapt to new goals makes it sensitive to noise, preventing the policy from settling into an equilibrium. In contrast, the DQN agent (purple bar) defied expectations of catastrophic interference, achieving the highest stability (*Bottom-Mid*) and smallest optimality gap (*Bottom-Left*). We interpret this stability not merely as efficient learning, but potentially as rigid exploitation. Lacking the entropy of SQL, DQN effectively ignores environmental noise, raising the possibility of *behavioral overfitting*, where the agent memorizes a conservative heuristic rather than learning a flexible map. Whether this strategy resembles biological habit formation or simply brittle memorization is a hypothesis we will explicitly test in future work in Goal 2 (Section 1.3) by aligning these policies with rodent tracking data. Finally, SQL (brown bar) presented a distinct trade-off. Its lower total and final reward (*Top-Mid* and *Top-Left*, respectively) compared to UVFA-DQN (light purple) and DQN (purple) reflects a necessary *entropy tax*: by optimizing for a distribution of paths, it sacrifices pure efficiency to maintain the behavioral variability characteristic of biological exploration.

Comparing MB planning and the SR in Figure 4.1 reveals an interesting trade-off between learning speed (*Top-Right*) and path efficiency (*Bottom-Left*). Dyna-Q agent with UCB (red bar) exploration achieved high convergence speeds (*Top-Right*) and rapid recovery times (*Bottom-Right*), comparable to the most Deep RL agents. However, this flexibility came at a severe cost: it suffered the worst total cumulative reward (*Top-Mid*) of all architectures. This highlights a significant drawback of using the UCB mechanism in this environment. Unlike maximum entropy control (brown bar), which merely softens the policy to maintain probabilistic flexibility, UCB adds a scalar bonus to unvisited states. In a task with static geometry but dynamic goals, this creates a compulsive *information-seeking* drive that forces the agent to constantly revisit low-value areas to reduce uncertainty, preventing it from settling into a stable, rewarding routine. Regarding the SR (pink bar), the results challenge the hypothesis that a predictive map is essential for this task as depicted in Section 4.1.7. Contrary to initial predictions of high instability, the SR (pink bar) agent maintained moderate stability (*Bottom-Mid*) and path efficiency (*Bottom-Left*), actually outperforming Dyna-Q (light green bar) in both metrics. However, it failed to provide a decisive advantage over the simpler tabular MF or Deep RL agents. This suggests that in a small state space with stochastic triggers, the computational overhead of maintaining a predictive map yields diminishing returns. A robust cached habit like DQN or a context-aware generalizer, such as UVFA, appears sufficient to solve the task effectively. This implies that solving such a task requires only computationally cheaper MF heuristics, particularly when the environment lacks the complex, deterministic sequential dependencies that would strictly require a predictive engine.

To ground our computational findings with biological findings, we juxtaposed the learning kinematics of our agents (Figure 4.2) against behavioral tracking data from mice performing an equivalent spatial navigation task. An implementation detail must be noted: a single trial in the biological protocol corresponds to one RL episode, whereas a full biological learning session equates to the single switch interval where the active goal is fixed (Table 3.1, Environment 5).

Figure 4.2 (*Top Row*) highlights the key characteristics of the biological phenotype: fast task acquisition and monotonic stability. Rodents achieve $> 80\%$ performance within just 20 trials (Figure 4.2A).

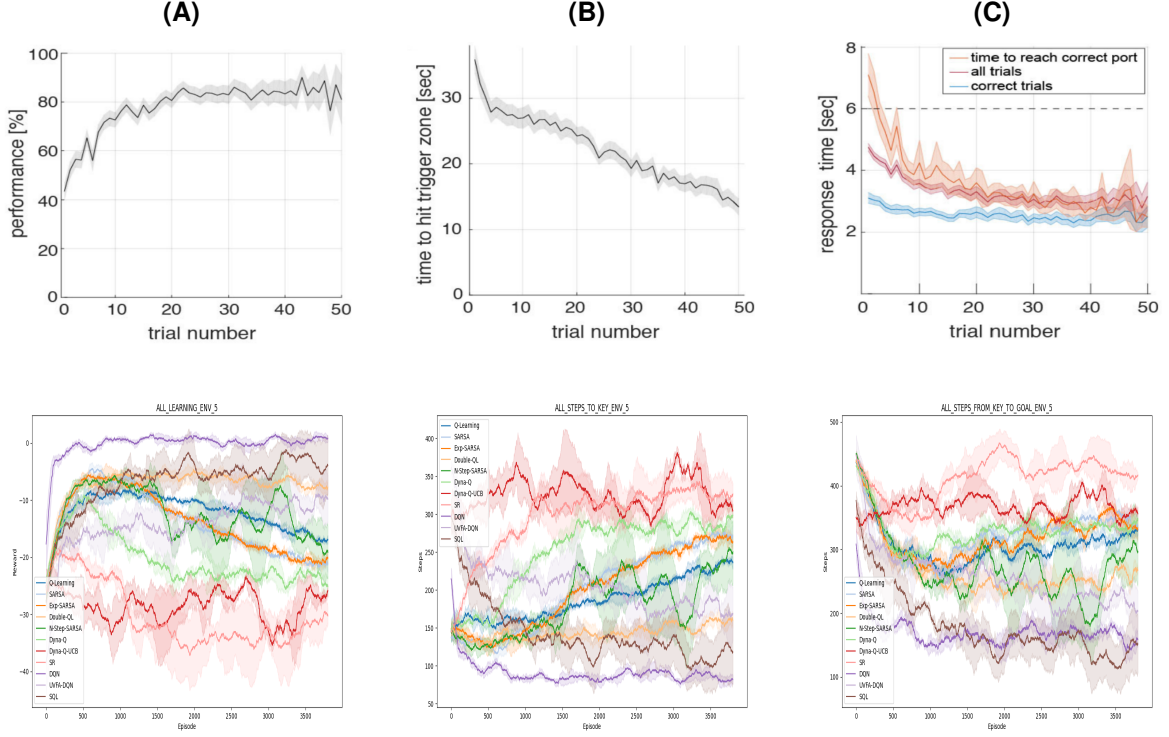


Figure 4.2: Comparative Learning Kinematics: Rodents and Artificial Agents. *Top Row:* Behavioral tracking data from mice ($n = 23$), adapted from Jercog et al. [20]. *Bottom Row:* Simulation results for all agent architectures in Environment 5. **(A)** Evolution of task success: Ratio of correct trials (*Top*) and Cumulative Reward (*Bottom*). **(B)** Search Efficiency: Time (*Top*) or steps (*Bottom*) required to locate the stochastic trigger zone. **(C)** Navigation Efficiency: Time (*Top*) or steps (*Bottom*) required to travel from the trigger zone to the goal. Data represents Environment 5 with a total of 4,000 episodes. Shaded regions indicate 95% confidence intervals (*Top*) or standard deviation across 3 independent runs (*Bottom*).

Additionally, they demonstrate a consistent, low-variance reduction in the time required to locate the trigger zone (Figure 4.2B), indicating the active optimization of an efficient search heuristic. Comparing this baseline to our artificial agents (*Bottom Row*) reveals that DQN (purple curve), SQL (brown curve) and UVFA-DQN (light purple curve) provide the best behavioral alignment (Figure 4.2A), particularly regarding search efficiency (Figure 4.2B). While MF tabular agents struggle with the high variance of random triggers, often overestimating the value of *lucky* transitions, Double Q-Learning (beige curve) mitigates this maximization bias, effectively filtering out noise to learn a robust scanning policy analogous to the mice.

Figure 4.2C reveals a clear divergence during the navigation phase. MF tabular and Dyna-Q agents suffered an unexpected performance decline over time, likely due to the constant overwriting of value tables during stochastic search. Finally, the n -Step SARSA agent (green curve) displays a *bumpy*, *U-shaped* profile, revealing that the accumulated prediction errors from extended lookahead eventually destabilized the policy. In contrast, the Deep RL and Double Q-Learning (beige curve) agents maintained a stable, flat baseline, demonstrating their ability to filter out noise and preserve long-term goal recall. We interpret this not merely as stability, but as behavioral overfitting: the neural network as well as decoupled update of UVFA-DQN, effectively ignores environmental noise to lock into a rigid search strategy. Conversely, the failure of the SR (pink curve) and Dyna-Q UCB (red curve) to converge on an

efficient path confirms that predictive complexity and compulsive exploration are counter-productive for simple, fixed-goal navigation. Specifically, while SR destabilized due to map invariance, Dyna-Q UCB failed due to a curiosity cost. In contrast to Dyna-Q UCB (red curve), SQL (brown curve) pays a minor *entropy tax* for flexibility without instability.

Based on the performance metrics of RL agents across Environments 1 through 6 shown in Figure B.2 in Appendix B, we can draw several conclusions regarding the trade-offs between habit formation, predictive mapping and generalized state representation with non-spatial context. Contrary to the hypothesis that complex exploration is unnecessary for simple tasks (Section 4.1.4 and 4.1.6), the Dyna-Q agent with UCB exploration demonstrated superior performance in Environments 1 and 2. It matched the high reward accumulation of Deep RL agents and exhibited fast recovery times. This suggests that in deterministic or periodic settings, the UCB bonus effectively drives the agent to verify environmental changes without getting stuck in sub-optimal local minima, outperforming the standard ϵ -greedy strategy used by Dyna-Q. Conversely, while Double Q-learning maintained high total reward, it struggled with cumulative reward accumulation during the periodic shifts of Environment 2, indicating that its cautious way of learning makes it too slow to react when the goal frequently jumps to a new location.

The behavior of the SR agent reveals a core weakness in predictive map-building (Section 4.1.7). In periodic contexts (Environment 2), although SR theoretically should adapt instantly by updating reward weights, it exhibited poor stability and convergence efficiency compared to Dyna-Q UCB or Deep RL agents. This implies that even minor shifts in the optimal path destabilize the successor matrix, forcing a noisy reconstruction of the predictive map. This failure was most pronounced in the hierarchical task (Environment 4), where the SR agent was the worst performer overall. Because SR encodes future occupancy given a specific policy, it cannot compress the distinct behavioral modes required for hierarchical tasks involving goals and subgoals into a single map, leading to high variance.

In Environments 5 and 6, the predictive engine hypothesis (Section 4.1.7) failed significantly. The SR agent consistently ranked among the worst in terms of reward and path efficiency, as attempting to learn a stable matrix in a random trigger zone environment proved computationally expensive and inaccurate. Although the SR agent quickly located the goal's vicinity, it failed to optimize the specific path, resulting in a fast yet *noisy* policy. Similarly, Dyna-Q UCB degraded in Environment 6. Without a stable model of how the key triggers the goal, its planning steps likely reinforced incorrect transitions, preventing the formation of a stable strategy. Ultimately, the only architectures to robustly solve these chaotic tasks were the Deep RL agents, such as DQN and UVFA, and SQL. This highlights that predicting future states is a fragile strategy in stochastic environments. Instead, the success of UVFA and DQN suggests that the hippocampus may rely on generalized state representations. By abstracting non-spatial context, such as goal vectors, it can instantly switch to the correct policy, rather than depending on brittle predictive maps.

Chapter 5

Conclusions and Future Directions

To achieve Goal 1 (Section 1.3), we established a computational baseline for complex navigation, modeled after the biological protocols of Jercog et al. [20]. We developed Environment 7 as an *in silico* replica of the stochastic *key-door* task used in rodent studies. By testing various RL agents in this setting, we successfully distinguished between simple habit formation and complex cognitive mapping.

Our results confirmed that while standard tabular MF mechanisms struggle with rapid re-learning they, and particularly *n*-Step SARSA, exhibited surprisingly robust reward performance, often outperforming the more complex MB and SR architectures in high-entropy regimes. This suggests a trade-off where simple agents succeed by *behaviorally overfitting*, calcifying a rigid, heuristic strategy that ignores noise, whereas planning agents proved inadequate by attempting to model the full stochastic structure of the task. Specifically, the SR agent struggled to stabilize its predictive map in the face of stochastic triggers, and Dyna-Q suffered from model noise, proving that attempting to derive state representations from predictability or full-blown simulation is computationally expensive and behaviorally brittle in this task. Instead, the superior recovery times of our Deep RL and maximum entropy agents suggest that the key to replicating biological plasticity in this particular protocol might be in the explicit encoding of non-spatial context such as goal vectors and including an entropy tax to enhance exploration of the trigger zone. This corroborates the view that abstracting task dynamics allows for better adaptation, whereas rebuilding a fragile map of specific transitions, or over-planning in a noisy world, remains insufficient for robust navigation.

Notably, our investigation into Goal 2 (Section 1.3) revealed a divergence between theoretical computational optimality and biological plausibility. While the SR is widely theorized as possible computational substrate of the hippocampus, our behavioral kinematic analysis indicated that it fails to capture the specific learning dynamics of mice in this task with several mnemonic demands. In the search phase (start $\rightarrow$ key), rodents demonstrate a consistent, monotonic reduction in the time required to locate the trigger, indicative of an optimized search heuristic. Here, the SR and standard MF tabular agents exhibited high variance and instability, struggling to process the stochastic trigger locations, whereas Deep RL agents and Double Q-Learning closely mirrored the rodent phenotype by effectively filtering noise to learn a robust search policy. A sharper contrast emerged in the navigation phase (key $\rightarrow$ goal): while *n*-Step

SARSA displayed a characteristic "U-shaped" valley and SR struggled with map stability, the biological data shows a flat, stable baseline of long-term memory in Deep RL agents. Hence, Deep RL and maximum entropy agents were the only architectures to reproduce this stability, suggesting they successfully *behaviorally fit* to the task structure.

These findings highlight the limitations of standard predictive implementations in highly stochastic goal dynamics. The failure of SR suggests that attempting to derive state representations solely from predictive transition maps is brittle in such dynamic environments. Conversely, the success of UVFA and DQN, which explicitly includes goal vectors, highlights the critical role of non-spatial context. Consequently, our next phase of research for this thesis must move beyond the view of the hippocampus as a purely euclidean cognitive map. Instead, we propose focusing on generalized state representations that integrate non-spatial features, such as topological connectivity and associative episodic memories. We propose validating these models in two stages: first by matching behavior, and then by aligning internal representations. By ensuring the agent first acts like the animal, we can then meaningfully test if its state representation match the hippocampus, creating a precise link between artificial and biological strategies.

To fully realize Goal 2 (Section 1.3), we will use behavioral cloning [85]. In this approach, we treat rodent tracking data as *expert demonstrations*, training our agents to strictly mimic the animals' paths. This allows us to test whether our architectures are complex enough to capture the specific navigation rules used by rodents. Finally, we will assess zero-shot flexibility by testing these pre-trained agents in new sessions. The goal is to see if they can instantly apply their prior knowledge to new problems, rather than relearning the environment from scratch.

Finally, to fulfill Goals 3 and 4 (Section 1.3), connecting our algorithms to physiology, we need a reliable method to characterize state representations. We will investigate the internal states of our best RL agents, addressing a critical deviation observed in our performance analysis: while Deep RL models achieve the closest behavioral fit to rodent trajectories in the Environments 5 and 7, the SR offers unique theoretical properties despite its practical instability in these tasks. Consequently, we will extract the hidden layers of Deep RL and SQL agents, and compare them against the predictive eigenvectors of the SR. Although the SR failed to solve the stochastic task in isolation, we hypothesize that hybridizing its predictive geometry with the non-spatial context of Deep RL, or augmenting it with MB planning, may resolve the trade-off between adaptability and stability as proposed in previous studies. Crucially, before attempting a direct correlation with biological data, we will benchmark these extracted representations against established theoretical frameworks, such as spectral graph representations and probabilistic inference cognitive maps [1, 19, 24]. We need this step because matching behavior does not prove matching representation. Even optimal agents often use internal maps that diverge from biological neural codes. Only after characterizing this theoretical geometry will we align these generalized state representations with high-density hippocampal recordings, aiming to determine if the brain utilizes a static map or a flexible, abstract state-space capable of integrating the complex rules of the task. See Figure D.1 for the project schedule.

Bibliography

- [1] R. V. Raju, J. S. Guntupalli, G. Zhou, C. Wendelken, M. Lázaro-Gredilla, and D. George. Space is a latent sequence: A theory of the hippocampus. *Sci. Adv.*, 10(31):eadm8470, Aug. 2024.
- [2] A. C. Schapiro, T. T. Rogers, N. I. Cordova, N. B. Turk-Browne, and M. M. Botvinick. Neural representations of events arise from temporal community structure. *Nat. Neurosci.*, 16(4):486–492, Apr. 2013.
- [3] G. Buzsáki and R. Llinás. Space and time in the brain. *Science*, 358(6362):482–485, Oct. 2017.
- [4] A. Shimbo, E.-I. Izawa, and S. Fujisawa. Scalable representation of time in the hippocampus. *Sci. Adv.*, 7(6):eabd7013, Feb. 2021.
- [5] I. Momennejad. Learning structures: Predictive representations, replay, and generalization. *Curr. Opin. Behav. Sci.*, 32:155–166, Apr. 2020.
- [6] M. M. Garvert, R. J. Dolan, and T. E. Behrens. A map of abstract relational knowledge in the human hippocampal-entorhinal cortex. *Elife*, 6, Apr. 2017.
- [7] Y. Niv. Learning task-state representations. *Nat. Neurosci.*, 22(10):1544–1553, Oct. 2019.
- [8] J. O’Keefe and L. Nadel. *The Hippocampus as a Cognitive Map*. Oxford University Press, London, England, Nov. 1978.
- [9] S. H. P. Collin, B. Milivojevic, and C. F. Doeller. Memory hierarchies map onto the hippocampal long axis in humans. *Nat. Neurosci.*, 18(11):1562–1564, Nov. 2015.
- [10] M. G. Mattar and N. D. Daw. Prioritized memory access explains planning and hippocampal replay. Nov. 2017.
- [11] I. Dasgupta and S. J. Gershman. Memory as a computational resource. *Trends Cogn. Sci.*, 25(3): 240–251, Mar. 2021.
- [12] S. Chandra, S. Sharma, R. Chaudhuri, and I. Fiete. Episodic and associative memory from spatial scaffolds in the hippocampus. *Nature*, 638(8051):739–751, Feb. 2025.
- [13] R. S. Sutton and A. G. Barto. *Reinforcement Learning*. Adaptive Computation and Machine Learning series. Bradford Books, Cambridge, MA, Feb. 1998.

- [14] E. M. Russek, I. Momennejad, M. M. Botvinick, S. J. Gershman, and N. D. Daw. Predictive representations can link model-based reinforcement learning to model-free mechanisms. *PLoS Comput. Biol.*, 13(9):e1005768, Sept. 2017.
- [15] Y. Liao, N. Yu, and J. Yan. A navigation path search and optimization method for mobile robots based on the rat brain's cognitive mechanism. *Biomimetics (Basel)*, 8(5):427, Sept. 2023.
- [16] W. de Cothi, N. Nyberg, E.-M. Griesbauer, C. Ghanamé, F. Zisch, J. M. Lefort, L. Fletcher, C. Newton, S. Renaudineau, D. Bendor, R. Grieves, É. Duvelle, C. Barry, and H. J. Spiers. Predictive maps in rats and humans for spatial navigation. *Curr. Biol.*, 32(17):3676–3689.e5, Sept. 2022.
- [17] C. C. Beron, S. Q. Neufeld, S. W. Linderman, and B. L. Sabatini. Mice exhibit stochastic and efficient action switching during probabilistic decision making. *Proc. Natl. Acad. Sci. U. S. A.*, 119(15):e2113961119, Apr. 2022.
- [18] S. J. Gershman. The successor representation: Its computational logic and neural substrates. *J. Neurosci.*, 38(33):7193–7200, Aug. 2018.
- [19] D. George, R. V. Rikhye, N. Gothoskar, J. S. Guntupalli, A. Dedieu, and M. Lázaro-Gredilla. Clone-structured graph representations enable flexible learning and vicarious evaluation of cognitive maps. *Nat. Commun.*, 12(1):2392, Apr. 2021.
- [20] L. Morales, D. P. Tomàs, J. Dalmau, J. de la Rocha, and P. E. Jercog. High-throughput task to study memory recall during spatial navigation in rodents. *Front. Behav. Neurosci.*, 14:64, May 2020.
- [21] E. C. Tolman. Cognitive maps in rats and men. *Psychol. Rev.*, 55(4):189–208, July 1948.
- [22] B. W. Balleine and J. P. O'Doherty. Human and rodent homologies in action control: corticostriatal determinants of goal-directed and habitual action. *Neuropsychopharmacology*, 35(1):48–69, Jan. 2010.
- [23] D. Aronov, R. Nevers, and D. W. Tank. Mapping of a non-spatial dimension by the hippocampal–entorhinal circuit. *Nature*, 543(7647):719–722, Mar. 2017.
- [24] H. Sanders, M. A. Wilson, and S. J. Gershman. Hippocampal remapping as hidden state inference. *Elife*, 9(e51140), June 2020.
- [25] M. Contreras, T. Pelc, M. Llofriu, A. Weitzenfeld, and J.-M. Fellous. The ventral hippocampus is involved in multi-goal obstacle-rich spatial navigation. *Hippocampus*, 28(12):853–866, Dec. 2018.
- [26] E. C. Tolman. *Purposive Behavior in Animals and Men*. Century Co., New York, 1932.
- [27] E. L. Thorndike. *Animal intelligence: An experimental study of the associative processes in animals*. Columbia University Press, New York, 1898.
- [28] A. Dickinson. Actions and habits: the development of behavioural autonomy. *Philosophical Transactions of the Royal Society of London. B, Biological Sciences*, 308(1135):67–78, 02 1985. ISSN 0080-4622. doi: 10.1098/rstb.1985.0010. URL <https://doi.org/10.1098/rstb.1985.0010>.

- [29] R. S. Sutton. Learning to predict by the methods of temporal differences. *Mach. Learn.*, 3(1):9–44, Aug. 1988.
- [30] R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the 7th International Conference on Machine Learning (ICML)*, pages 216–224, 1990.
- [31] R. S. Sutton. Planning by incremental dynamic programming. In *Machine Learning Proceedings 1991*, pages 353–357. Elsevier, 1991.
- [32] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, Dec. 2020.
- [33] W. Carvalho, M. S. Tomov, W. de Cothi, C. Barry, and S. J. Gershman. Predictive representations: Building blocks of intelligence. *Neural Comput.*, 36(11):2225–2298, Oct. 2024.
- [34] E. L. Thorndike. The law of effect. *Am. J. Psychol.*, 39(1/4):212, Dec. 1927.
- [35] Z. J. Pierce-Messick and L. H. Corbit. Stimulus conditions that promote habitual control. *Sci. Rep.*, 14(1):30303, Dec. 2024.
- [36] C. D. Adams. Variations in the sensitivity of instrumental responding to reinforcer devaluation. *Q. J. Exp. Psychol. B*, 34(2b):77–98, May 1982.
- [37] R. M. J. Deacon and J. N. P. Rawlins. T-maze alternation in the rodent. *Nat. Protoc.*, 1(1):7–12, 2006.
- [38] R. Morris. Developments of a water-maze procedure for studying spatial learning in the rat. *J. Neurosci. Methods*, 11(1):47–60, May 1984.
- [39] E. C. Tolman and C. H. Honzik. *Introduction and removal of reward and maze performance in rats, by E. C. Tolman and C. H. Honzik ...* University of California publications in psychology v. 4, no. 17. University of California Press, Berkeley, Calif, 1930.
- [40] L. Traetta. At the beginning of learning studies there was the maze. *Open J. Med. Psychol.*, 09(04): 168–183, 2020.
- [41] B. E. Pfeiffer and D. J. Foster. Hippocampal place-cell sequences depict future paths to remembered goals. *Nature*, 497(7447):74–79, May 2013.
- [42] J. Webb, P. Steffan, B. Y. Hayden, D. Lee, C. Kemere, and M. McGinley. Foraging animals use dynamic bayesian updating to model meta-uncertainty in environment representations. *PLoS Comput. Biol.*, 21(4):e1012989, Apr. 2025.
- [43] A. Johnson, Z. Varberg, J. Benhardus, A. Maahs, and P. Schrater. The hippocampus and exploration: dynamically evolving behavior and neural representations. *Front. Hum. Neurosci.*, 6:216, July 2012.

- [44] J. Grau-Moya, F. Leibfried, and P. Vrancx. Soft q-learning with mutual-information regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyEtjoCqFX>.
- [45] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine. Reinforcement learning with deep energy-based policies. 2017.
- [46] T. M. George, W. de Cothi, K. L. Stachenfeld, and C. Barry. Rapid learning of predictive maps with STDP and theta phase precession. *Elife*, 12(e80663), Mar. 2023.
- [47] J. Huxter, N. Burgess, and J. O’Keefe. Independent rate and temporal coding in hippocampal pyramidal cells. *Nature*, 425(6960):828–832, Oct. 2003.
- [48] W. D. Penny, P. Zeidman, and N. Burgess. Forward and backward inference in spatial cognition. *PLoS Comput. Biol.*, 9(12):e1003383, Dec. 2013.
- [49] E. A. Ludvig, R. S. Sutton, and E. J. Kehoe. Evaluating the TD model of classical conditioning. *Learn. Behav.*, 40(3):305–319, Sept. 2012.
- [50] W. Schultz, P. Dayan, and P. R. Montague. A neural substrate of prediction and reward. *Science*, 275(5306):1593–1599, Mar. 1997.
- [51] C. B. Holroyd and M. G. H. Coles. The neural basis of human error processing: Reinforcement learning, dopamine, and the error-related negativity. *Psychol. Rev.*, 109(4):679–709, 2002.
- [52] P. W. Glimcher. Understanding dopamine and reinforcement learning: the dopamine reward prediction error hypothesis. *Proc. Natl. Acad. Sci. U. S. A.*, 108 Suppl 3(supplement_3):15647–15654, Sept. 2011.
- [53] I. Momennejad, E. M. Russek, J. H. Cheong, M. M. Botvinick, N. D. Daw, and S. J. Gershman. The successor representation in human reinforcement learning. *Nat. Hum. Behav.*, 1(9):680–692, Sept. 2017.
- [54] P. R. Montague and T. J. Sejnowski. The predictive brain: temporal coincidence and temporal order in synaptic learning mechanisms. *Learning & Memory*, 1(1):1–33, 1994.
- [55] M. J. Frank, L. C. Seeberger, and R. C. O’reilly. By carrot or by stick: cognitive reinforcement learning in parkinsonism. *Science*, 306(5703):1940–1943, Dec. 2004.
- [56] J. P. O’Doherty, S. W. Lee, and D. McNamee. The structure of reinforcement-learning mechanisms in the human brain. *Curr. Opin. Behav. Sci.*, 1:94–100, Feb. 2015.
- [57] E. Tricomi, B. W. Balleine, and J. P. O’Doherty. A specific role for posterior dorsolateral striatum in human habit learning. *Eur. J. Neurosci.*, 29(11):2225–2232, June 2009.
- [58] E. Massi, J. Barthélemy, J. Mailly, R. Dromnelle, J. Canitrot, E. Poniatowski, B. Girard, and M. Khamassi. Model-based and model-free replay mechanisms for reinforcement learning in neurorobotics. *Front. Neurobot.*, 16:864380, June 2022.

- [59] I. K. Brunec and I. Momennejad. Predictive representations in hippocampal and prefrontal hierarchies. *J. Neurosci.*, 42(2):299–312, Jan. 2022.
- [60] J. Gläscher, N. Daw, P. Dayan, and J. P. O’Doherty. States versus rewards: dissociable neural prediction error signals underlying model-based and model-free reinforcement learning. *Neuron*, 66(4):585–595, May 2010.
- [61] M. Khamassi and M. D. Humphries. Integrating cortico-limbic-basal ganglia architectures for learning model-based and model-free navigation strategies. *Front. Behav. Neurosci.*, 6:79, Nov. 2012.
- [62] I. Momennejad, A. R. Otto, N. D. Daw, and K. A. Norman. Offline replay supports planning in human reinforcement learning. *Elife*, 7, Dec. 2018.
- [63] B. B. Doll, K. D. Duncan, D. A. Simon, D. Shohamy, and N. D. Daw. Model-based choices involve prospective neural activity. *Nat. Neurosci.*, 18(5):767–772, May 2015.
- [64] M. Keramati, P. Smittenaar, R. J. Dolan, and P. Dayan. Adaptive integration of habits into depth-limited planning defines a habitual-goal-directed spectrum. *Proc. Natl. Acad. Sci. U. S. A.*, 113(45):12868–12873, Nov. 2016.
- [65] S. W. Lee, S. Shimojo, and J. P. O’Doherty. Neural computations underlying arbitration between model-based and model-free learning. *Neuron*, 81(3):687–699, Feb. 2014.
- [66] P. Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993. doi: 10.1162/neco.1993.5.4.613.
- [67] T. J. Madarasz. Better transfer learning with inferred successor maps. 2019.
- [68] M. R. Mehta, M. C. Quirk, and M. A. Wilson. Experience-dependent asymmetric shape of hippocampal receptive fields. *Neuron*, 25(3):707–715, Mar. 2000.
- [69] W. de Cothi and C. Barry. Neurobiological successor features for spatial navigation. *Hippocampus*, 30(12):1347–1355, Dec. 2020.
- [70] J. H. Sul, H. Kim, N. Huh, D. Lee, and M. W. Jung. Distinct roles of rodent orbitofrontal and medial prefrontal cortex in decision making. *Neuron*, 66(3):449–460, May 2010.
- [71] J. Bono, S. Zannone, V. Pedrosa, and C. Clopath. Learning predictive cognitive maps with spiking neurons during behavior and replays. *Elife*, 12(e80671), Mar. 2023.
- [72] G. Pezzulo, M. A. A. van der Meer, C. S. Lansink, and C. M. A. Pennartz. Internally generated sequences in learning and executing goal-directed behavior. *Trends Cogn. Sci.*, 18(12):647–657, Dec. 2014.
- [73] M. H. Plitt and L. M. Giocomo. Experience-dependent contextual codes in the hippocampus. *Nat. Neurosci.*, 24(5):705–714, May 2021.

- [74] E. R. Wood, P. A. Dudchenko, R. J. Robitsek, and H. Eichenbaum. Hippocampal neurons encode information about different types of memory episodes occurring in the same location. *Neuron*, 27(3):623–633, Sept. 2000.
- [75] R. M. Grieves, E. R. Wood, and P. A. Dudchenko. Place cells on a maze encode routes rather than destinations. *Elife*, 5, June 2016.
- [76] C. J. C. H. Watkins and P. Dayan. Q-learning. *Mach. Learn.*, 8(3-4):279–292, May 1992.
- [77] H. Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010. URL https://proceedings.neurips.cc/paper_files/paper/2010/file/091d584fced301b442654dd8c23b3fc9-Paper.pdf.
- [78] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, Feb. 2015.
- [79] T. Schaul, D. Horgan, K. Gregor, and D. Silver. Universal value function approximators. 07 2015.
- [80] H. van Seijen, H. van Hasselt, S. Whiteson, and M. Wiering. A theoretical and empirical analysis of expected sarsa. In *2009 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*, pages 177–184, 2009. doi: 10.1109/ADPRL.2009.4927542.
- [81] H. van Seijen, H. van Hasselt, S. Whiteson, and M. Wiering. A theoretical and empirical analysis of expected sarsa. In *2009 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning*, pages 177–184, 2009. doi: 10.1109/ADPRL.2009.4927542.
- [82] H. Robbins and S. Monro. A stochastic approximation method. *Ann. Math. Stat.*, 22(3):400–407, Sept. 1951.
- [83] R. S. Sutton. Integrated modeling and control based on reinforcement learning and dynamic programming. In R. Lippmann, J. Moody, and D. Touretzky, editors, *Advances in Neural Information Processing Systems*, volume 3. Morgan-Kaufmann, 1990. URL https://proceedings.neurips.cc/paper_files/paper/1990/file/d9fc5b73a8d78fad3d6dffe419384e70-Paper.pdf.
- [84] S. Thrun and A. Schwartz. Issues in using function approximation for reinforcement learning. 10 1993.
- [85] D. J. Foster, A. Block, and D. Misra. Is behavior cloning all you need? understanding horizon in imitation learning. 2024.

Appendix A

Gridworld Behavioral Analysis

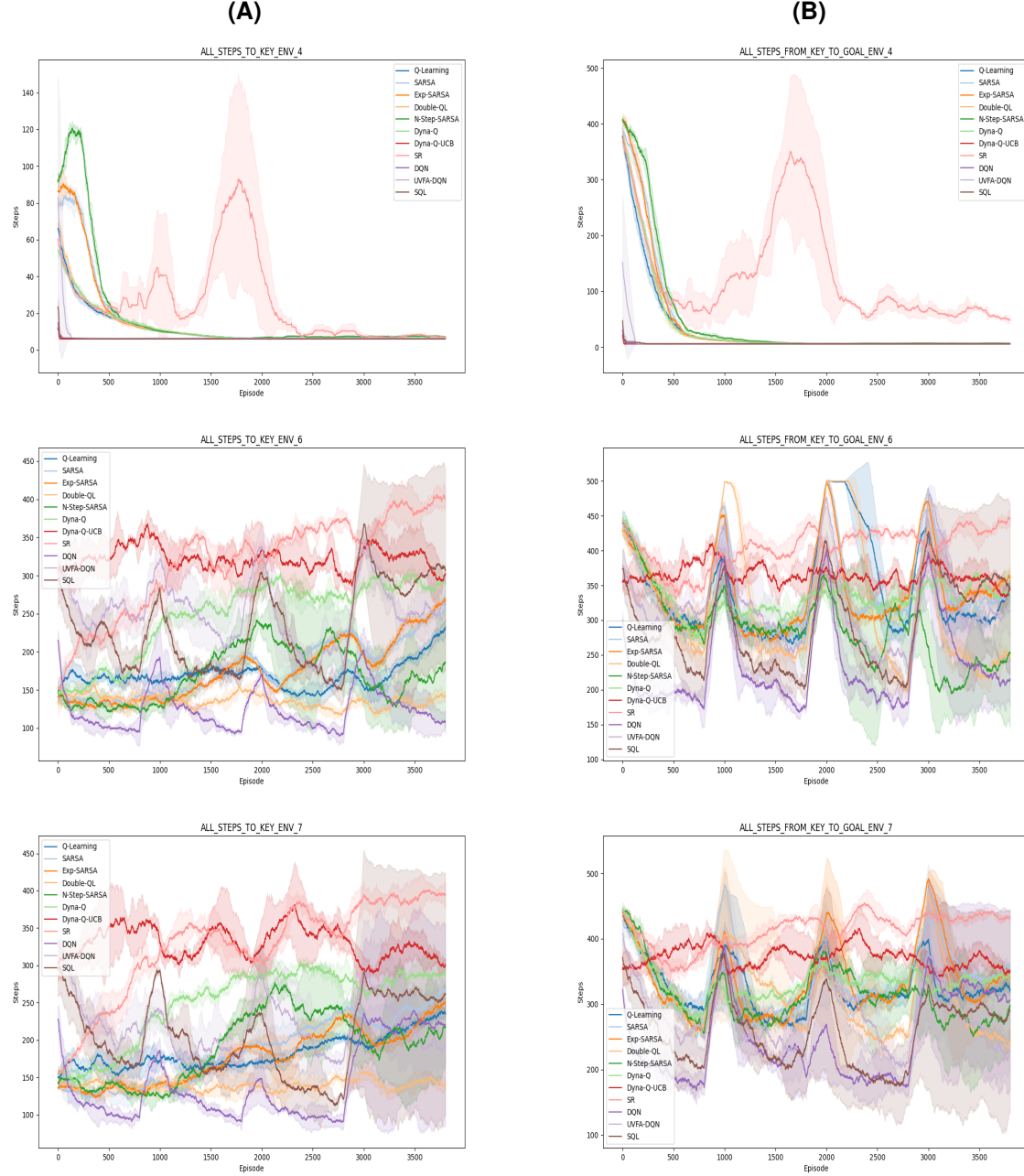


Figure A.1: MF, MB and SR Kinematics: Search and Navigation. Comparative analysis of behavioral components in Environments 4, 6 and 7. (A) Search Phase: The number of steps required to locate the trigger zone from the start position. (B) Goal Phase: The number of steps required from the trigger zone to the active reward location. The reward relocates among the four corners every 1000 episodes. Plots reflect data averaged over 3 independent runs with shaded regions indicating standard deviation.

Appendix B

Agent Performance Benchmarking

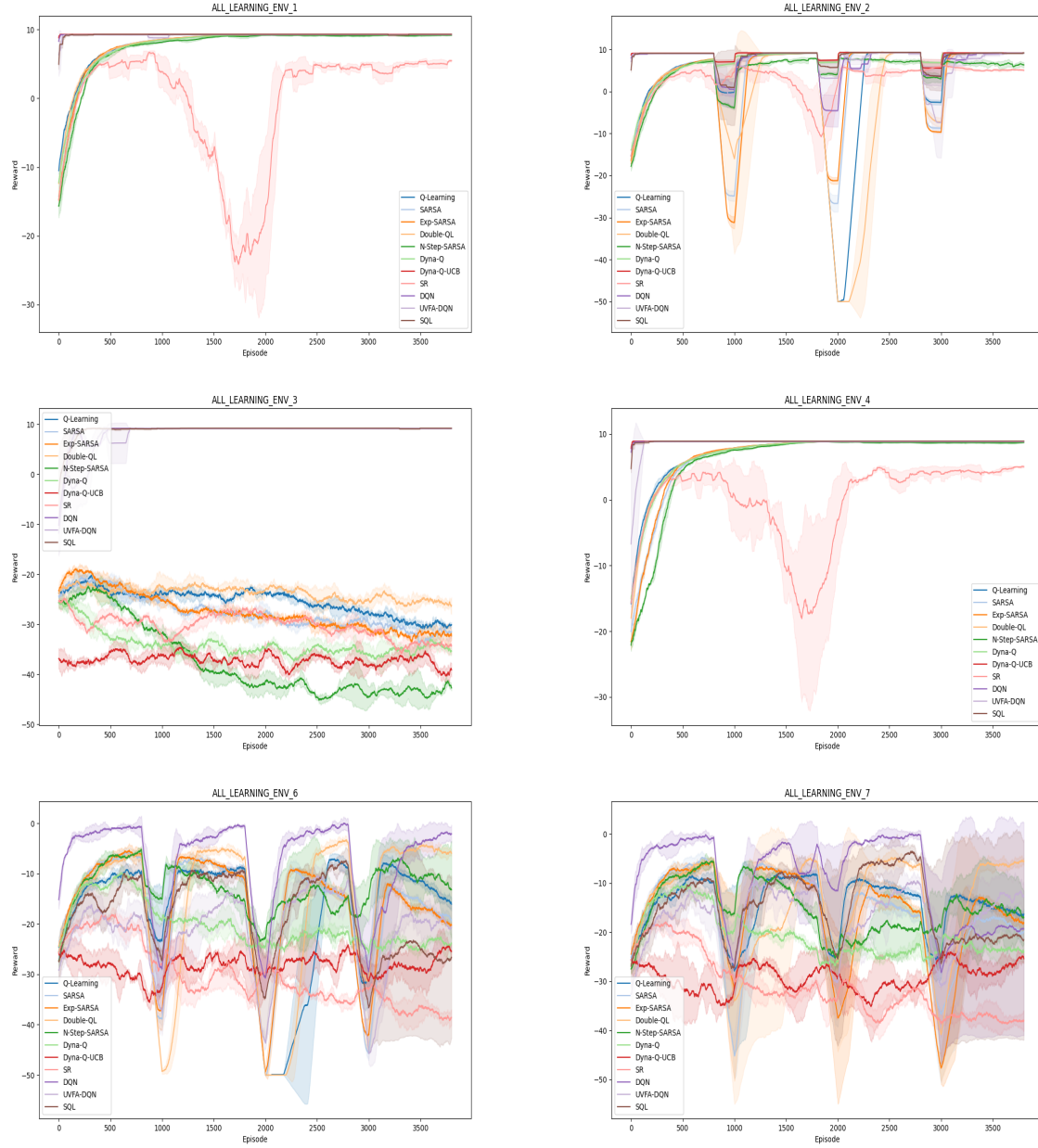


Figure B.1: Comparative Learning Dynamics of MF, MB, and SR Agents. Temporal evolution of cumulative reward against MF, MB and SR architectures across distinct environmental configurations. Plots show the moving average of rewards over 4000 episodes with a active reward switch interval of 1000 episodes, averaged across 3 independent runs, with shaded regions indicating standard deviation.

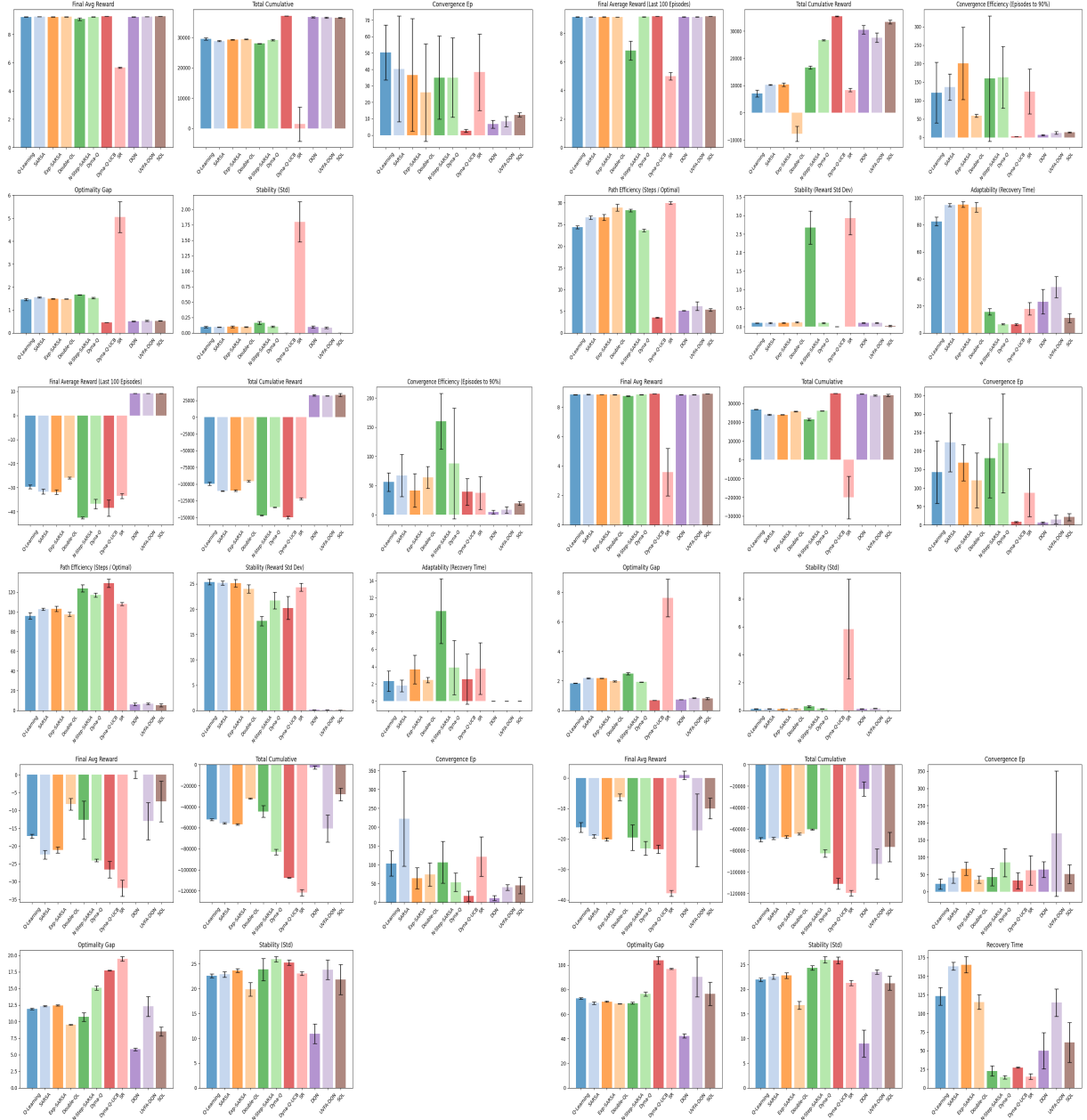


Figure B.2: Quantitative Performance Metrics for MF, MB and SR Agents. Performance metrics extracted from learning sessions across Environments 1 through 6, benchmarking the distinct control policies of MF, MB and SR architectures. Error bars represent standard error across 3 independent runs.

Appendix C

Model Parameters

Table C.1: Global Simulation Parameters. Configuration applied across all experimental runs.

Parameter	Value
Total Episodes	4000
Independent Runs (N)	3
Moving Average Window	200 episodes
Switch Interval	1000 episodes
Maximum Steps per Episode	500
Reward Structure	+10.0 at goal, -0.1 otherwise

Table C.2: Agent-Specific Hyperparameters. Configuration for Tabular and Deep MF RL agents.

Category	Algorithm	Parameter	Value/Description
Tabular	Q-Learning, SARSA, Expected-SARSA, Double QL	α	0.1
		γ	0.99
		ϵ	$1.0 \rightarrow 0.05$ (Linear annealing over ≈ 2000 eps)
		Initialization	$Q(s, a) = 0.0$
	n -Step SARSA	n	5
Deep RL	DQN	Memory Buffer	Local trajectory storage cleared per episode
		Architecture	FC: $[Input \rightarrow 128 \rightarrow 128 \rightarrow Actions]$ (ReLU)
		Learning Rate	ADAM 1×10^{-3}
		Batch Size	64
		τ	0.005
		Loss Function	SmoothL1Loss (Huber)
	UVFA-DQN	Architecture	Two-Stream: State $[128, 64]$, Goal $[128, 64]$
		Combined Head	$[128 \rightarrow 128 \rightarrow Actions]$
		Learning Rate	ADAM 3×10^{-4}
		Batch Size	256
	SQL	Architecture	FC: $[Input \rightarrow 256 \rightarrow 256 \rightarrow Actions]$
		Entropy (α)	AUTO-TUNED Target: $-0.1 \log(1/ A)$
		τ	0.05
		Batch Size	256
Common	All Deep Agents	Replay Buffer	Size: 100,000 AND Uniform Sampling

Table C.3: Agent-Specific Hyperparameters. Configuration for Dyna-Q, Dyna-Q (UCB) and SR agents.

Category	Algorithm	Parameter	Value/Description
Model-Based	Dyna-Q (Epsilon)	α	0.1
		γ	0.99
		ϵ	0.1 (FIXED)
		Planning Steps (N)	10
		Model Type	Deterministic
	Dyna-Q (UCB)	c	0.01
		N	10
		Action Selection	UCB1
		Bonus	$c\sqrt{2\ln(t)/N(s,a)}$
	Predictive Map	SR	α
α_r			0.1
γ			0.99
ϵ			0.1 (FIXED)
Representation			$M(s, s', a)$ (TABULAR)
Value			$Q(s, a) = \sum_{s'} M(s, s', a) \cdot w(s')$

Appendix D

Schedule Plan

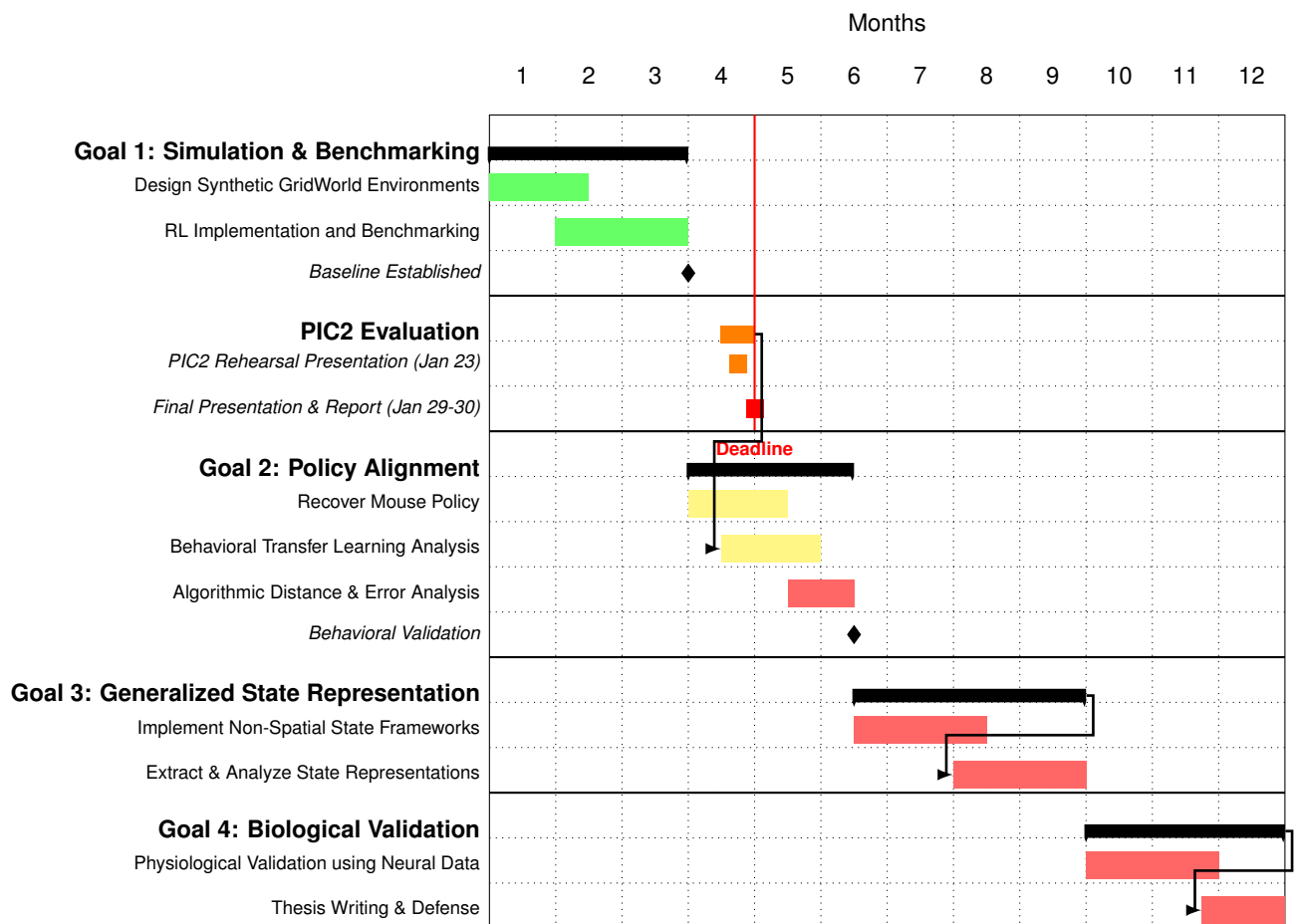


Figure D.1: 12-Month Research Schedule. Green: Completed tasks. Orange/Red Marks: Evaluation Deadlines. Yellow: Ongoing tasks. Red: Future tasks.

Appendix E

Implementations

Algorithm 1 Q-Learning

```
1: Initialize  $Q(s, a)$  arbitrarily for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ , with  $Q(\text{terminal}, \cdot) = 0$ 
2: Parameters: step size  $\alpha \in (0, 1]$ , discount  $\gamma \in [0, 1)$ , exploration rate  $\epsilon > 0$ 
3: for each episode do
4:   Initialize  $S$ 
5:   repeat
6:     Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
7:     Take action  $A$ , observe  $R, S'$ 
8:     Update Q-value:
9:        $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
10:     $S \leftarrow S'$ 
11:   until  $S$  is terminal
12: end for
```

Algorithm 2 SARSA

```
1: Initialize  $Q(s, a)$  arbitrarily for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ , with  $Q(\text{terminal}, \cdot) = 0$ 
2: Parameters: step size  $\alpha \in (0, 1]$ , discount  $\gamma \in [0, 1)$ , exploration rate  $\epsilon > 0$ 
3: for each episode do
4:   Initialize  $S$ 
5:   Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
6:   repeat
7:     Take action  $A$ , observe  $R, S'$ 
8:     Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
9:     Update Q-value:
10:     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ 
11:     $S \leftarrow S'; A \leftarrow A'$ 
12:   until  $S$  is terminal
13: end for
```

Algorithm 3 Expected SARSA

```
1: Initialize  $Q(s, a)$  arbitrarily,  $\forall s \in \mathcal{S}, a \in \mathcal{A}$ 
2: Parameters: step size  $\alpha$ , discount  $\gamma$ , exploration  $\epsilon$ 
3: for each episode do
4:   Initialize  $S$ 
5:   repeat
6:     Choose  $A$  from  $S$  using policy  $\pi$  derived from  $Q$ 
7:     Take action  $A$ , observe  $R, S'$ 
8:      $V_{S'} = \sum_a \pi(a|S')Q(S', a)$ 
9:      $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma V_{S'} - Q(S, A)]$ 
10:     $S \leftarrow S'$ 
11:   until  $S$  is terminal
12: end for
```

Algorithm 4 n -step SARSA

```
1: Initialize  $Q(s, a)$  arbitrarily,  $\forall s \in \mathcal{S}, a \in \mathcal{A}$ 
2: Parameters: step size  $\alpha$ , discount  $\gamma$ , exploration  $\epsilon$ , step horizon  $n$ 
3: for each episode do
4:   Initialize  $S_0 \neq \text{terminal}$ 
5:   Select  $A_0$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
6:    $T \leftarrow \infty$ 
7:   for  $t = 0, 1, 2, \dots$  do
8:     if  $t < T$  then
9:       Take action  $A_t$ , observe  $R_{t+1}, S_{t+1}$ 
10:      if  $S_{t+1}$  is terminal then
11:         $T \leftarrow t + 1$ 
12:      else
13:        Select  $A_{t+1}$  using policy derived from  $Q$ 
14:      end if
15:    end if
16:     $\tau \leftarrow t - n + 1$ 
17:    if  $\tau \geq 0$  then
18:       $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$ 
19:      if  $\tau + n < T$  then
20:         $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ 
21:      end if
22:       $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$ 
23:    end if
24:    if  $\tau = T - 1$  then break
25:  end if
26: end for
27: end for
```

Algorithm 5 Double Q-Learning

```
1: Initialize  $Q^A(s, a)$  and  $Q^B(s, a)$  arbitrarily,  $\forall s \in \mathcal{S}, a \in \mathcal{A}$ 
2: Parameters: step size  $\alpha \in (0, 1]$ , discount  $\gamma$ , exploration  $\epsilon$ 
3: for each episode do
4:   Initialize  $S$ 
5:   repeat
6:     Choose  $A$  from  $S$  using policy derived from  $Q^A + Q^B$  (e.g.,  $\epsilon$ -greedy)
7:     Take action  $A$ , observe  $R, S'$ 
8:     Select update target randomly (e.g., coin flip)
9:     if Update  $Q^A$  then
10:       $a^* \leftarrow \operatorname{argmax}_a Q^A(S', a)$ 
11:       $Q^A(S, A) \leftarrow Q^A(S, A) + \alpha [R + \gamma Q^B(S', a^*) - Q^A(S, A)]$ 
12:    else
13:       $b^* \leftarrow \operatorname{argmax}_a Q^B(S', a)$ 
14:       $Q^B(S, A) \leftarrow Q^B(S, A) + \alpha [R + \gamma Q^A(S', b^*) - Q^B(S, A)]$ 
15:    end if
16:     $S \leftarrow S'$ 
17:  until  $S$  is terminal
18: end for
```

Algorithm 6 Deep Q-Learning with Experience Replay

```
1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize sequence  $s_1 = \{x_1\}$  and preprocess  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$ 
9:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
13:    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
14:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to  $\theta$ 
15:    Every  $C$  steps reset  $\hat{Q} = Q$ 
16:  end for
17: end for
```

Algorithm 7 UVFA-DQN: Two-Stream Architecture

```
1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize separate state network  $\phi$ , goal network  $\psi$  and head network  $f$ 
3: Initialize target networks  $\phi^-, \psi^-, f^-$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Sample goal  $g$  from goal space  $\mathcal{G}$ 
6:   Initialize state  $s_1$ 
7:   for  $t = 1, T$  do
8:     Compute embeddings:  $z_s = \phi(s_t), z_g = \psi(g)$ 
9:     Compute values:  $Q(s_t, \cdot, g) = f([z_s, z_g])$ 
10:    With probability  $\epsilon$  select random action  $a_t$ , else  $a_t = \arg\max_a Q$ 
11:    Execute action  $a_t$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
12:    Store transition  $(s_t, a_t, r_t, s_{t+1}, g)$  in  $D$ 
13:    Sample minibatch  $(s_j, a_j, r_j, s_{j+1}, g_j)$  from  $D$ 
14:    Calculate target:
15:       $z'_s = \phi^-(s_{j+1}), z'_g = \psi^-(g_j)$ 
16:       $y_j = \begin{cases} r_j & \text{if terminal } s_{j+1} \\ r_j + \gamma \max_{a'} f^-([z'_s, z'_g]) & \text{otherwise} \end{cases}$ 
17:      Perform gradient descent on  $(y_j - Q(s_j, a_j, g_j; \theta))^2$  w.r.t.  $\theta$ 
18:      Soft update target parameters  $\theta^-$  toward  $\theta$ 
19:   end for
20: end for
```

Algorithm 8 Discrete Soft Q-Learning

```
1: Initialize replay memory  $D$ 
2: Initialize Soft Q-networks  $Q_{\theta_1}, Q_{\theta_2}$  (to mitigate overestimation)
3: Initialize target entropy  $\bar{\mathcal{H}}$  and log-temperature  $\alpha$ 
4: for episode = 1,  $M$  do
5:   Initialize state  $s$ 
6:   for  $t = 1, T$  do
7:     Compute probabilities:  $\pi(a|s) \propto \exp(Q_{\theta_1}(s, a)/\alpha)$ 
8:     Sample action  $a \sim \pi(\cdot|s)$ 
9:     Execute  $a$ , observe  $r, s'$ 
10:    Store  $(s, a, r, s')$  in  $D$ 
11:    Update Q-functions:
12:    Sample batch  $B = \{(s, a, r, s')\}$  from  $D$ 
13:    Calculate soft value:
14:       $V(s') = \alpha \log \sum_{a'} \exp\left(\frac{\min_{j=1,2} Q_{\theta_j}(s', a')}{\alpha}\right)$ 
15:    Calculate target:  $y = r + \gamma V(s')$ 
16:    Descent on  $\sum_{j=1,2} (y - Q_{\theta_j}(s, a))^2$ 
17:    Update Temperature  $\alpha$ :
18:    Descent on dual loss:  $-\alpha(\bar{\mathcal{H}} - \mathcal{H}(\pi(\cdot|s)))$ 
19:    Soft update target networks
20:     $s \leftarrow s'$ 
21:   end for
22: end for
```

Algorithm 9 Dyna-Q with UCB / ϵ -greedy

```
1: Initialize  $Q(s, a)$ ,  $\text{Model}(s, a)$ , and visit counts  $N(s, a)$ 
2: Parameters:  $\alpha, \gamma$ , planning steps  $N$ , UCB constant  $c$ 
3: for each episode do
4:   Initialize  $S$ 
5:   repeat
6:     if using UCB then
7:        $A \leftarrow \operatorname{argmax}_a \left( Q(S, a) + c \sqrt{2 \ln(\sum N(S, \cdot)) / N(S, a)} \right)$ 
8:     else
9:       With probability  $\epsilon$  choose random  $A$ , else  $A \leftarrow \operatorname{argmax}_a Q(S, a)$ 
10:    end if
11:    Take action  $A$ , observe  $R, S'$ 
12:     $N(S, A) \leftarrow N(S, A) + 1$ 
13:    Direct RL:  $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
14:    Model Learning:  $\text{Model}(S, A) \leftarrow (R, S')$ 
15:    Planning:
16:    for  $i = 1$  to  $N$  do
17:       $S_{sim} \leftarrow$  random previously visited state
18:       $A_{sim} \leftarrow$  random action previously taken in  $S_{sim}$ 
19:       $R_{sim}, S'_{sim} \leftarrow \text{Model}(S_{sim}, A_{sim})$ 
20:       $Q(S_{sim}, A_{sim}) \leftarrow Q(S_{sim}, A_{sim}) + \alpha[R_{sim} + \gamma \max_a Q(S'_{sim}, a) - Q(S_{sim}, A_{sim})]$ 
21:    end for
22:     $S \leftarrow S'$ 
23:  until  $S$  is terminal
24: end for
```

Algorithm 10 Successor Representation

```
1: Initialize SR tensor  $M(a, s, s')$  as identity matrices
2: Initialize reward weights  $w(s)$  as zeros
3: Parameters: learning rate  $\alpha$ , discount  $\gamma$ , exploration  $\epsilon$ 
4: for each episode do
5:   Initialize state  $s$ 
6:   repeat
7:     Action Selection:
8:     Compute  $Q(s, a) = M(a, s, :) \cdot w$  for all  $a$ 
9:     With prob  $\epsilon$  choose random  $a$ , else  $a \leftarrow \operatorname{argmax}_a Q(s, a)$ 
10:    Execute  $a$ , observe reward  $r$  and next state  $s'$ 
11:    Update Reward Weights ( $w$ ):
12:     $w(s') \leftarrow w(s') + \alpha(r - w(s'))$ 
13:    Update Successor Representation ( $M$ ):
14:    Select next action  $a'$  greedily:  $a' \leftarrow \operatorname{argmax}_a (M(a, s', :) \cdot w)$ 
15:    Define current occupancy vector  $\phi(s)$  (one-hot at  $s$ )
16:    Compute TD Target:  $\hat{M} \leftarrow \phi(s) + \gamma M(a', s', :)$ 
17:    Update:  $M(a, s, :) \leftarrow M(a, s, :) + \alpha(\hat{M} - M(a, s, :))$ 
18:     $s \leftarrow s'$ 
19:  until  $s$  is terminal
20: end for
```
